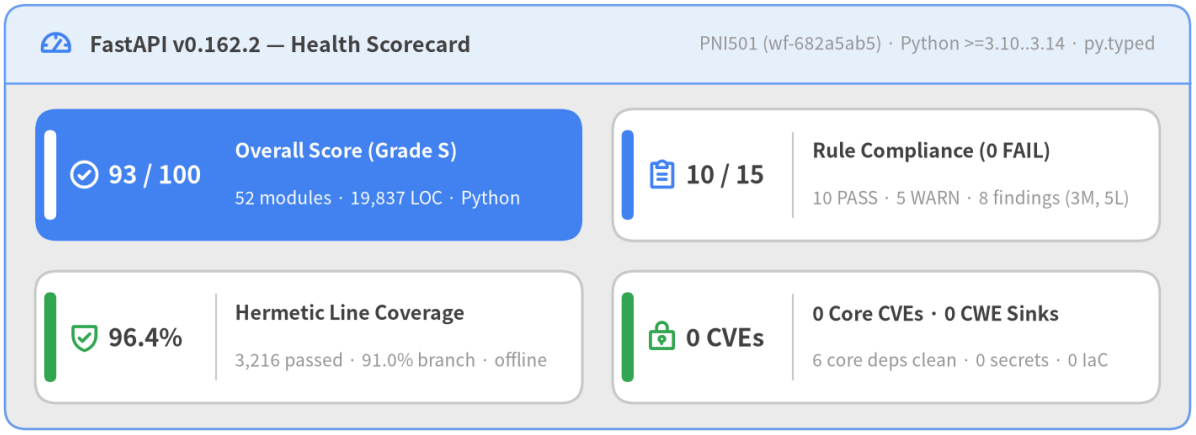
PNI Software Health Diagnostic Report

FastAPI `v0.162.2` — *Automated Architecture, Static AST, Hermetic Runtime & Supply-Chain Diagnostic*

This independent technical showcase evaluates **FastAPI** `v0.162.2` (created by Sebastián Ramírez / `@tiangolo`, MIT License) across 7 software engineering dimensions using **PNI501** and **Drawlib**. All upstream framework modules, test suites, and dependency manifests are inspected unmodified in zero-pollution hermetic environments.



FastAPI v0.162.2 — PNI501 Overall Engineering Health Scorecard

Diagnostic Attribute	Verified Metadata & Audit Baseline
Target Project	`fastapi` (`v0.162.2` , tag `3adc94ef` , Python `>=3.10..3.14` , packaged PEP 561 `fastapi/py.typed`)
Target Language & Profile	`python` (`library` — High-performance ASGI 3.0 web API framework & OpenAPI 3.1 compiler)
Upstream License & Attribution	MIT License (`LICENSE-FASTAPI` ; Copyright © Sebastián Ramírez — FastAPI® is a registered trademark of Sebastián Ramírez)
Diagnostic Workflow	`pni501` (Workflow Run ID `wf-682a5ab5` , category `audit`)
Evaluated At	`2026-11-08T12:21:21+00:00` (`2026-11-08T12:21:21.280043+00:00`)
Overall Score & Grade	**93 / 100 (Grade S)** — `10 / 15` rules `PASS` , `5 WARN` , `0 FAIL` (`8` findings: `0` CRITICAL, `0` HIGH, `3` MEDIUM, `5` LOW)

Diagnostic Attribute	Verified Metadata & Audit Baseline
Analyzed Codebase Scope	`52` source modules (`19,837` LOC, `121` classes, `438` functions) & `273` core test modules (`3,216` hermetic tests passed under `--network=none`)

> **Unofficial Engineering Showcase Notice**: This diagnostic report is an independent automated analysis generated with **PNI501** and **Drawlib** to celebrate FastAPI's engineering design and is not affiliated with or endorsed by Sebastián Ramírez or the official **FastAPI project**.

Table of Contents

1. Executive Summary

2. Architecture & Source Code Structure

3. Recommended Remediation Roadmap

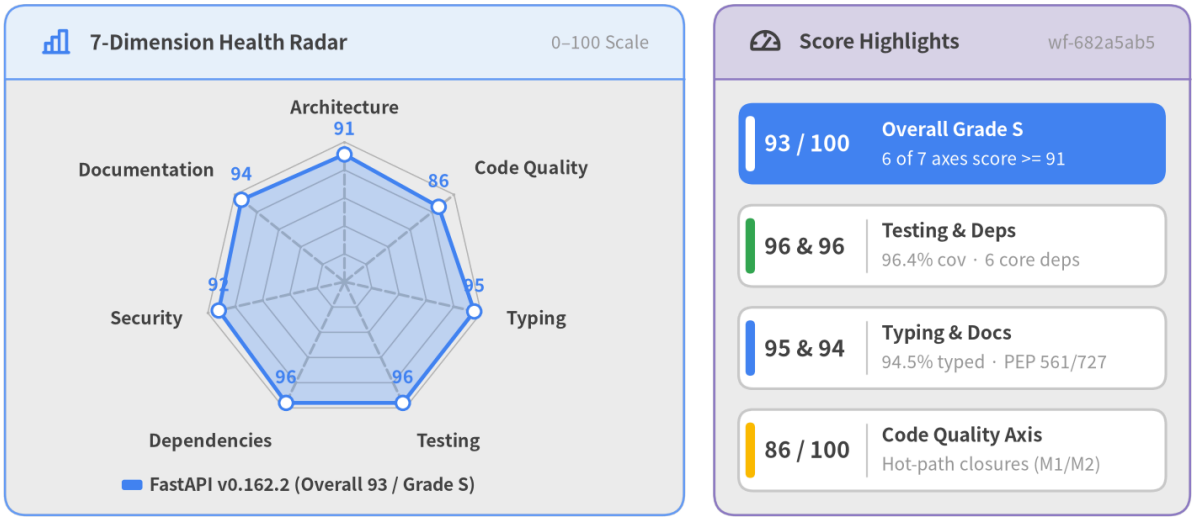
4. Audit Standards, Rule Compliance & Findings

5. Deep Dive: Code Quality, Testing & Runtime

6. Deep Dive: Security, Supply Chain & API Surface

1. Executive Summary

FastAPI `v0.162.2` achieves an overall PNI501 engineering health score of **93/100 (Grade S)** with **10/15** rules passed (**5 WARN**, **0 FAIL**) across **52** Python source modules (**19,837 LOC**, **121** classes, **438** functions). Hermetic `--network=none` container testing confirms **3,216** passing tests (**0** failures), **96.4%** line coverage, and **91.0%** branch coverage, while AST and Trivy scans verify **94.5%** callable type coverage, **zero** dangerous CWE execution sinks, **zero** hardcoded secrets, and **zero** CVEs in core runtime dependencies.



FastAPI v0.162.2 — 7-Dimension PNI501 Engineering Health Radar & Headline Score Profile

1.1 Seven-Dimension Engineering Health Breakdown

Six of FastAPI's seven evaluated dimensions achieve **Grade S** (`>= 90`), anchored by **Testing** (`96`), **Dependencies** (`96`), **Typing** (`95`), and **Documentation** (`94`). **Code Quality** (`86` , **Grade A)** reflects an intentional architectural trade-off in FastAPI's core compiler: pre-building per-route request handling and OpenAPI synthesis inside dense closures so runtime ASGI dispatch remains fast and declarative.

Dimension	Score	Grade	Engineering Rationale & Verified Evidence
Architecture	**91 / 100**	**S**	Cohesive 5-tier downward architecture across `52` source modules (`19,837` LOC) bridging Starlette ASGI and Pydantic v2, with `2` cross-tier cycles guarded under `TYPE_CHECKING` and only `1` localized Tier-1 runtime cycle (`_compat.v2 <-> params -> datastructures`).
Code Quality	**86 / 100**	**A**	High API ergonomics, zero wildcard imports, and zero global mutable state, balanced against concentrated branching complexity inside hot-path request compilation (`get_request_handler` , cognitive= `159`) and OpenAPI path synthesis (`get_openapi_path` , cognitive= `109`).
Typing	**95 / 100**	**S**	`94.5%` callable signature type coverage across `438` functions, packaged PEP 561 `fastapi/py.typed` marker, PEP 484 explicit `X as X` re-exports across `20` public symbols, and dual `mypy` strict + Astral `ty` verification.
Testing	**96 / 100**	**S**	`96.4%` measured line coverage (`4,628 / 4,801` statements) and `91.0%` branch coverage across `3,216` passing core unit and ASGI `TestClient` tests inside an air-gapped (`--network=none`) non-root container with `filterwarnings=['error']` .
Dependencies	**96 / 100**	**S**	Ultra-lean `6` -package core PEP 621 dependency footprint with flexible `>=` SemVer lower bounds, `0` unpinned specs, isolated PEP 735 `dependency-groups` , and a deterministic `uv.lock` .
Security	**92 / 100**	**S**	Zero CVEs across all `6` core runtime dependencies, zero hardcoded or committed secrets, zero container IaC misconfigurations, and zero dangerous CWE code sinks across `19,837` LOC; `1` HIGH transitive `dnspython@2.8.0` advisory exists in dev/optional `uv.lock` .
Documentation	**94 / 100**	**S**	Pioneering PEP 727 `Annotated[..., Doc()]` inline parameter documentation across all public APIs with zero docstring-to-signature mismatches and zero empty docstrings.

1.2 Audit Step Baseline, Hermetic Runtime & Supply-Chain Headlines

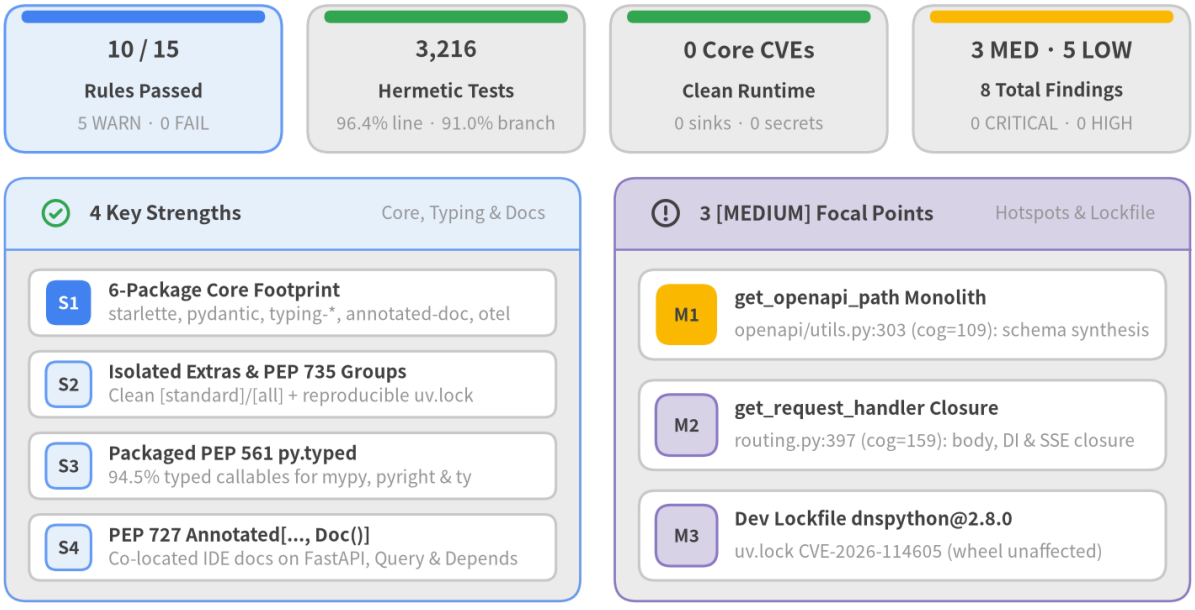
Across all six executed PNI501 audit stages (detailed in **Chapter 4**), zero rules failed (`10 PASS` , `5 WARN` , `0 FAIL`), yielding `8` consolidated location findings (`0` CRITICAL, `0` HIGH, `3` MEDIUM, `5` LOW):

Lowest-Scoring Audit Steps (Ascending)	Score & Grade	Rule Outcomes	Primary Diagnostic Signal
`python_ast_audit`	**89 / 100 (A)**	`4 PASS`, `3 WARN`, `0 FAIL`	`94.5%` type coverage and `0` CWE sinks across `52` files; warns on deliberate hot-path cognitive complexity (`get_request_handler=159`, `get_openapi_path=109`), `1` Tier-1 import cycle, and `1` mutable default (`values={}`).
`security_vulnerability_audit`	**91 / 100 (S)**	`0 PASS`, `2 WARN`, `0 FAIL`	`0` core runtime CVEs, `0` secrets, `0` IaC misconfigs, and `0` CWE sinks; warns on `7` transitive dev/optional `uv.lock` advisories (`1` HIGH: `CVE-2026-114605`) and ephemeral test-harness OS base-image packages.
`repo_structure_audit`	**93 / 100 (S)**	`4 PASS`, `1 WARN`, `0 FAIL`	Exemplary `6`-package PEP 621 manifest, `uv.lock`, and `fastapi/py.typed`; standalone showcase snapshot omits repository-level `SECURITY.md` and Dependabot config.

*(The remaining three audit steps scored **94/100 (S)** on `library_hygiene_audit`, **95/100 (S)** on `python_test_audit`, and **97/100 (S)** on `container_runtime_audit`.)*

Verification Domain	Gate Status	Key Quantitative Metrics
Container Runtime Verification	`passed`	Harness build `passed` (`0.79s` cached, Python `3.12`, non-root UID `10001`); air-gapped `--network=none` pytest `passed` with `3,216 / 3,228` tests passing (`0` failed, `0` errors, `12` skipped in `265.57s`); `96.4%` **line** & `91.0%` **branch** coverage; library import smoke check `passed`.
Security & Supply-Chain Scan	`failed` *(lockfile/OS)*	`0` **core runtime CVEs** across all `6` production dependencies; `app_cves=7` (`0` CRITICAL, `1` **HIGH** `CVE-2026-114605` in optional transitive `dnspython@2.8.0` -> `2.9.0`, `5` MEDIUM, `1` LOW) in dev/optional `uv.lock`; `misconfigs=0`; `secrets=0`; `ast_dangerous_sinks=0`.

1.3 Key Engineering Strengths vs. Architectural Focal Points



FastAPI v0.162.2 — Headline Health KPIs, Key Engineering Strengths & [MEDIUM] Focal Points

Verified Engineering Strengths

1. **Ultra-Lean 6-Package Core Dependency Footprint**: `pyproject.toml` restricts runtime dependencies to `starlette>=0.46.0`, `pydantic>=2.9.0`, `typing-extensions>=4.8.0`, `typing-inspection>=0.4.2`, `annotated-doc>=0.0.2`, and `opentelemetry-api>=1.44.0`, keeping downstream installation surfaces minimal and free of upper-bound version deadlocks.
2. **Clean Separation of Optional Extras & PEP 735 Dependency Groups**: Optional user-facing bundles (`[standard]`, `[all]`) are cleanly decoupled from development workflows (`tests`, `docs`, `dev` under `[dependency-groups]`), backed by a deterministic `uv.lock`.
3. **Packaged PEP 561 `fastapi/py.typed` Marker**: Combined with `94.5%` callable annotation completeness and explicit `from ... import X as X` re-exports in `fastapi/__init__.py`, downstream applications benefit from first-class `mypy`, `pyright`, and `ty` static type checking.

4. **Pioneering PEP 727 `Annotated[..., Doc()]` Inline Documentation**: Public constructors and parameter factories (`FastAPI` , `APIRouter` , `Path` , `Query` , `Body` , `Header` , `Cookie` , `Depends` , `Security`) co-locate structured documentation directly inside type annotations, preventing signature-to-docstring drift (`0` mismatches across `52` modules).

Prioritized `[MEDIUM]` Focal Points & Remediation Outlook

Zero `CRITICAL` or `HIGH` findings exist in FastAPI's source code or core runtime dependencies. The three `[MEDIUM]` findings represent targeted maintainability and lockfile hygiene opportunities (mapped to concrete actions in Chapter 3 and Chapter 6):

1. `[MEDIUM]` `common/maintainability` — `get_openapi_path` (`fastapi/openapi/utils.py:303`): Merges operation parameters, request bodies, security scopes, callbacks, and multi-status response schemas inside a single `230` -line synthesis routine (`cognitive=109`). Factoring response and callback builders into composable helpers will simplify isolated unit maintenance.
2. `[MEDIUM]` `common/maintainability` — `get_request_handler` (`fastapi/routing.py:397`): Embeds body parsing (JSON, multipart, form, raw), `solve_dependencies` execution, endpoint dispatch, streaming/SSE branching, and response validation inside a single `408` -line nested closure (`cognitive=159`). Extracting internal stage helpers while retaining route-registration pre-computation preserves FastAPI's hot-path ASGI throughput.
3. `[MEDIUM]` `common/security_vulnerability` — **Transitive** `dnspython@2.8.0` **in** `uv.lock` (`CVE-2026-114605`): Affected by DNS compression pointer traversal Denial of Service (CVSS `7.5` HIGH; fixed in `dnspython>=2.9.0`), pulled transitively by optional `email-validator` in the development lockfile. Core `fastapi` wheel installations are unaffected because `pyproject.toml` does not pin `dnspython` .

2. Architecture & Source Code Structure

FastAPI `v0.162.2` organizes **52 Python source modules** (**19,837 LOC**, **121 classes**, **438 functions**) into a disciplined **5-Tier Declarative ASGI Compiler & Type-Driven Adapter** architecture (**91/100**, Grade **S**). Rather than relying on import-time global registries or reflection-heavy request dispatch, FastAPI compiles standard Python type annotations and PEP 727 `Annotated[..., Doc()]` metadata at route-registration time into pre-built `Dependant` directed acyclic graphs (DAGs), Pydantic v2 `ModelField` validators, and asynchronous Starlette ASGI closures.

FastAPI v0.162.2 — Physical Package Layout & Subsystem Inventory (52 Modules · 19,837 LOC)

Package Tree (fastapi/) PEP 561 py.typed

```
fastapi/ (52 files · 19,837 LOC)
├── applications.py (4,118 LOC)
├── routing.py (5,726 LOC)
├── param_functions.py (2,430 LOC)
├── params.py (763 LOC)
├── encoders.py & sse.py (523 LOC)
├── dependencies/ (3 · 1,487 LOC)
│   └── models.py & utils.py
├── openapi/ (5 · 1,291 LOC)
│   └── utils.py, models.py, docs.py
├── security/ (7 · 1,294 LOC)
├── telemetry/ (4 · 869 LOC)
├── _compat/ (3 · 617 LOC)
├── middleware/ (7 · 24 LOC)
└── py.typed & Starlette facades
```

Subsystem Inventory by Tier LOC & Core Symbols

Module / Subpackage	Tier	LOC	Key Types & Responsibility
applications.py	T5	4,118	FastAPI app, lifespan & docs routes
param_functions.py	T5	2,430	Query, Path, Body, Depends, Doc()
routing.py	T4	5,726	APIRouter, APIRoute, SSE/WS engine
openapi/ (5)	T4	1,291	OpenAPI 3.1.0, Swagger UI & ReDoc
sse.py & handlers	T4	218	EventSourceResponse & 422 JSON
dependencies/ (3)	T3	1,487	Dependant & solve_dependencies()
security/ (7)	T3	1,294	OAuth2, HTTPBearer, APIKey, OIDC
telemetry/ & mw/	T2	924	NativeTelemetry & AsyncExitStack
params.py & core	T1	1,566	Param descriptors, encoders, utils
_compat/ (3)	T1	617	Pydantic v2 ModelField & schemas

FastAPI v0.162.2 Source Package Tree & Subsystem LOC Distribution (52 Modules, 19,837 LOC)

2.1 System Profile, Core Dependency Stack & External Integrations

FastAPI operates purely as an embeddable **library runtime** (`runtime_types = ["library"]` , Python `>=3.10..3.14`) with a packaged PEP 561 `fastapi/py.typed` marker for strict downstream static type checking (`mypy` , `pyright` , and `ty`). Its core installation surface is intentionally constrained to **six SemVer-floored runtime packages** (`0` unpinned specifiers, `0` core runtime CVEs), delegating transport servers, multipart parsers, and alternative JSON serializers to optional extras (`[standard]` , `[all]`) and PEP 735 `[dependency-groups]` .

Core Dependency (`pyproject.toml`)	Version Floor	Architectural Role in FastAPI `v0.162.2`	Primary Consumer Mo
`starlette`	`>=0.46.0`	Asynchronous ASGI 3.0 lifecycle, HTTP/WebSocket routing primitives, middleware wrapping, `Request` / `Response` , `BackgroundTasks` , and `TestClient` .	`applications.py` , `routing.py` , `middlewa` `responses.py`
`pydantic` *(+ `pydantic-core`)*	`>=2.9.0`	Rust-backed `TypeAdapter` validation, response serialization, composite body models, and `GenerateJsonSchema` OpenAPI `$defs` generation.	`_compat/v2.py` , `_compat/shared.py` , `params.py` , `openapi/models.py`
`typing-inspection`	`>=0.4.2`	High-speed runtime introspection of `Annotated[T, ...]` qualifiers, `Union` branches, and generic type aliases during route compilation.	`dependencies/utils.p` `_compat/v2.py`
`typing-extensions`	`>=4.8.0`	Backported typing constructs (`Annotated` , `TypedDict` , `Literal` , `override`) ensuring uniform behavior across Python `3.10` through `3.14` .	`types.py` , `params.py` , `param_functions.py` , `applications.py`
`annotated-doc`	`>=0.0.2`	PEP 727 `Doc("...")` inline parameter documentation co-located directly inside public callable signatures for rich IDE hover tooltips.	`applications.py` , `routing.py` , `param_functions.py`
`opentelemetry-api`	`>=1.44.0`	Zero-vendor-lock-in distributed tracing spans, route-template attributes, runtime lifespan hooks, and exception telemetry recording.	`telemetry/_asgi.py` , `telemetry/_runtime.p` `telemetry/_api.py`

External Integration Boundaries

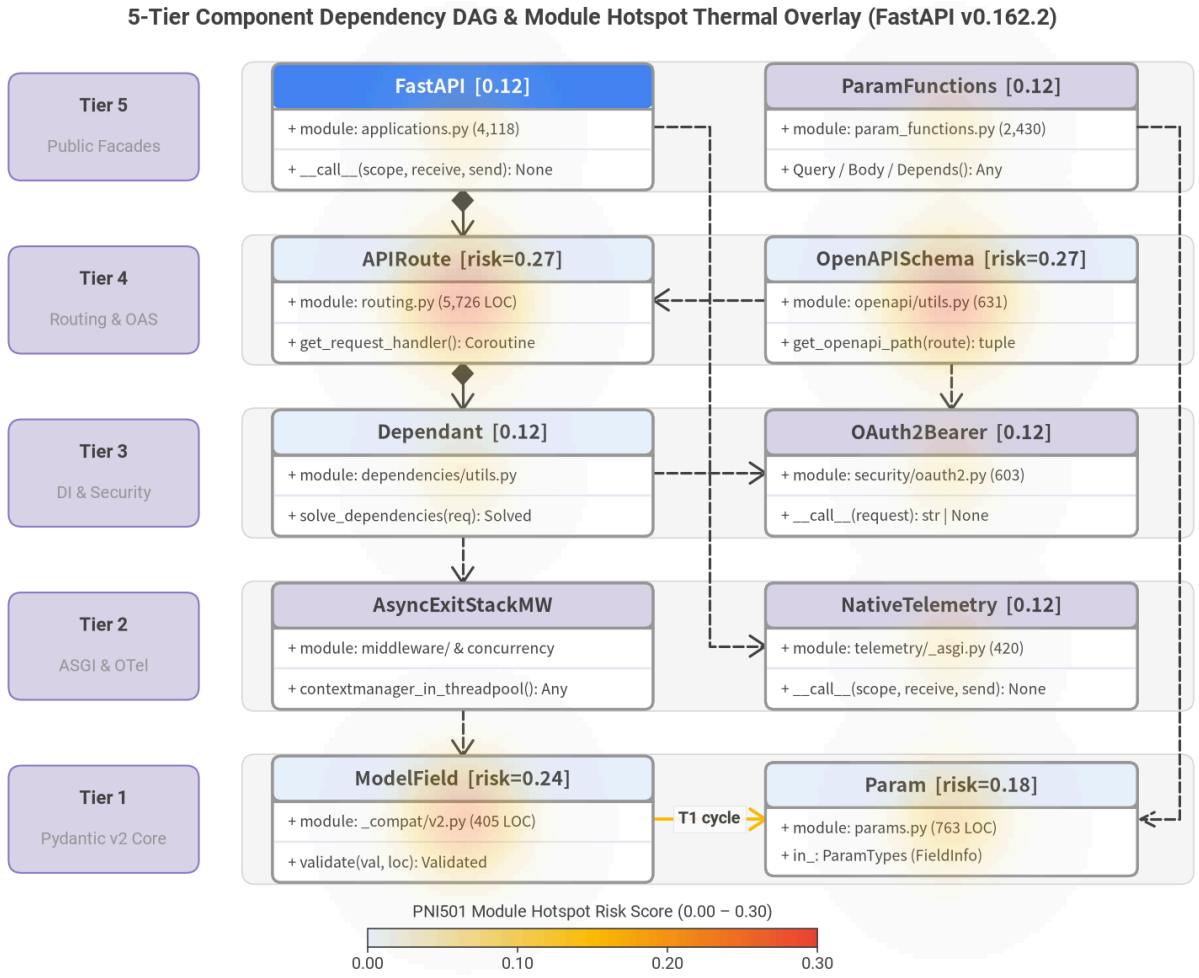
1. **ASGI 3.0 Application Servers**: Uvicorn, Hypercorn, Granian, or Fastar invoke `FastAPI.__call__(scope, receive, send)` directly with zero framework-specific server

coupling.

2. **Starlette ASGI Subsystems**: `FastAPI` subclasses `starlette.applications.Starlette`, composing Starlette's `Router`, lifespan context managers, `ServerErrorMiddleware`, `ExceptionMiddleware`, and thin developer re-export facades (`requests.py`, `websockets.py`, `staticfiles.py`, `templating.py`, `testclient.py`).
3. **Pydantic v2 Core Schema Engine**: `fastapi/_compat/v2.py` encapsulates `ModelField`, `TypeAdapter`, and `GenerateJsonSchema`, insulating routing and dependency resolution from internal Pydantic API churn.
4. **OpenTelemetry SDK & OTLP Exporters**: `fastapi/telemetry/_asgi.py` (`NativeTelemetry`, `ExceptionTelemetryMiddleware`) and `_runtime.py` emit low-overhead spans and exception events whenever an OpenTelemetry tracer/meter provider is configured.
5. **Interactive API Documentation Bundles**: `fastapi/openapi/docs.py` serves self-contained HTML entrypoints for Swagger UI (`/docs`, plus OAuth2 redirect support) and ReDoc (`/redoc`) bound to the lazy-synthesized `/openapi.json` document.

2.2 Five-Tier Declarative Architecture & Boundary Assessment

FastAPI enforces a strict top-down compilation and execution pipeline across **five architectural tiers**, progressing from public developer facades (`Tier 5`) down to Pydantic v2 schema/validation descriptors (`Tier 1`). Figure 2.2 overlays PNI501's deterministic `module_hotspots` risk scores (`0.12` to `0.27`) directly onto the five-tier UML component dependency graph.



5-Tier Component Dependency DAG & Thermal Risk Score Overlay (routing.py=0.27, openapi/utils.py=0.27, _compat/v2.py=0.24)

Five-Tier Layer Specifications

Layer	Module Paths (`52` Total)	Allowed Downward Dependencies	Core Architectural Responsibilities
Tier 5: Public Application & Developer Facades *(12 files)*	`fastapi/__init__.py`, `__main__.py`, `cli.py`, `applications.py`, `param_functions.py`, `requests.py`, `responses.py`, `websockets.py`, `staticfiles.py`, `templating.py`, `testclient.py`, `background.py`	Tiers `4`, `3`, `2`, `1`	Top-level `FastAPI` ASGI application class, PEP 727 `Doc()`-annotated parameter factory functions (`Query`, `Path`, `Body`, `Header`, `Cookie`, `Form`, `File`, `Depends`, `Security`), CLI stub, and ergonomic Starlette re-export facades.

Layer	Module Paths (`52` Total)	Allowed Downward Dependencies	Core Architectural Responsibilities
Tier 4: Routing, Streaming & OpenAPI Synthesis *(8 files)*	`fastapi/routing.py`, `sse.py`, `openapi/__init__.py`, `openapi/utils.py`, `openapi/docs.py`, `openapi/models.py`, `openapi/constants.py`, `exception_handlers.py`	Tiers `3`, `2`, `1`	Compiles endpoint callables into Starlette ASGI handlers (`APIRouter`, `APIRoute`, `APIWebSocketRoute`), native Server-Sent Events (`EventSourceResponse`, `ServerSentEvent`) & JSONL streaming, HTTP/WebSocket `422` exception handlers, and OpenAPI `3.1.0` / Swagger UI / ReDoc generation.
Tier 3: Dependency Injection & Security Subsystems *(10 files)*	`fastapi/dependencies/{__init__,utils,models}.py`, `fastapi/security/{__init__,oauth2,http,api_key,open_id_connect_url,base,utils}.py`	Tiers `2`, `1`	Introspects callable signatures via `typing-inspection` into recursive `Dependant` DAGs, resolves per-request sub-dependencies with memoization and `AsyncExitStack` yield teardown, and enforces OAuth2, HTTP Basic/Bearer/Digest, APIKey, and OpenID Connect security schemes.
Tier 2: ASGI Middleware, Concurrency & Native Telemetry *(12 files)*	`fastapi/middleware/{__init__,asyncexitstack,cors,gzip,httpsredirect,trustedhost,wsgi}.py`, `concurrency.py`, `fastapi/telemetry/{__init__,_asgi,_runtime,_api}.py`	Tier `1`	Orchestrates request-scoped `AsyncExitStackMiddleware` cleanup, synchronous contextmanager worker-thread offloading (`contextmanager_in_threadpool`), Starlette middleware re-exports, and native OpenTelemetry span/exception instrumentation (`NativeTelemetry`).
Tier 1: Pydantic v2 Adapter, Descriptors & Core Types *(10 files)*	`fastapi/_compat/{__init__,v2,shared}.py`, `params.py`, `datastructures.py`, `encoders.py`, `exceptions.py`, `utils.py`, `types.py`, `logger.py`	*None (`[]` — Leaf Foundation)*	Encapsulates Pydantic v2 `ModelField` / `TypeAdapter` validation and `GenerateJsonSchema` hooks, `FieldInfo` parameter descriptors (`Param` hierarchy), `UploadFile` / `DefaultPlaceholder` data structures, `jsonable_encoder`, and core exception hierarchies.

Architectural Boundary & Coupling Assessment

Static AST import-graph verification across all `52` source modules confirms strong downward architectural discipline:

- **Cleanly Guarded Cross-Tier Type Cycles (`2` , Zero Runtime Impact)**: Two upward/peer type-annotation references— `fastapi.dependencies.utils -> fastapi.utils -> fastapi.routing -> fastapi.dependencies.utils` and `fastapi.routing <->`

`fastapi.utils` —are strictly isolated inside `if TYPE_CHECKING:` blocks, preventing import-time circularity while preserving full static type precision.

- **Localized Intra-Tier-1 Runtime Cycle (`1`)**: Exactly one runtime circular import chain exists inside the foundational leaf tier: `fastapi._compat.v2 -> fastapi.params -> fastapi.datastructures -> fastapi._compat.v2` (driven by runtime `Param` / `UploadFile` / `DefaultPlaceholder` cross-checks). No runtime cycles cross tier boundaries.
- **Private Submodule Boundary Crossings (`4`)**: `fastapi/applications.py`, `fastapi/routing.py`, and `fastapi/background.py` perform four internal imports directly from private underscore modules (`fastapi.telemetry._asgi`, `fastapi.telemetry._runtime`, and `fastapi.telemetry._api`) rather than routing through a package-level `fastapi/telemetry/__init__.py` facade.
- **Zero Global Mutable State (`0`)**: All route trees, OpenAPI schemas, and `dependency_overrides` live on explicit `FastAPI` or `APIRouter` instances, allowing multiple independent applications to coexist cleanly in the same Python process.

2.3 Pre-Sorted Module Hotspots & Source Module Inventory

Pre-Sorted Module Hotspots (`module_hotspots`)

PNI501 ranks repository targets by composite static complexity, maintainability, typing, and supply-chain finding density. In FastAPI `v0.162.2`, maximum module risk caps at a modest `0.27` (zero `CRITICAL` or `HIGH` source code findings), concentrated in the two intentional compilation/synthesis workhorses (`fastapi/routing.py` and `fastapi/openapi/utils.py`):

Rank	Target Path	LOC	`risk_score`	Findings	Triggered Rules	Primary Engineering Driver
1	`fastapi/routing.py`	`5,726`	`0.27`	`1 medium`	`common/maintainability`	`get_request_handler` (`cognitive=159`, `408` LOC) unifies body parsing, `solve_dependencies()`, endpoint execution, SSE/JSONL streaming, and response serialization in one hot-path closure.

Rank	Target Path	LOC	risk_score	Findings	Triggered Rules	Primary Engineering Driver
2	fastapi/openapi/utils.py	631	0.27	1 medium	common/maintainability	get_openapi_path (cognitive=109, 230 LOC) merges parameters, request bodies, OAuth2 scopes, callbacks, and status responses in one synthesis pass.
3	fastapi/_compat/v2.py	405	0.24	2 low	common/architecture, python/code_quality	Participates in the intra-Tier-1 runtime import cycle with params.py / datastructures.py and defines a values: dict[str, Any] = {} default in ModelField.validate.
4	fastapi/params.py	763	0.18	1 low	python/typing	Descriptor __init__ signatures omit explicit -> None return annotations across Param / FieldInfo subclasses (Path.in_ untyped class attribute).
5	uv.lock	0	0.15	1 medium	common/security_vulnerability	Transitive optional/dev lockfile entry dnspython@2.8.0 carries CVE-2026-114605 (fixed in 2.9.0; core fastapi installs are unaffected).
6	fastapi/applications.py	4,118	0.12	0	—	Extensive inline PEP 727 Doc() reference documentation across FastAPI.__init__ and HTTP verb decorators; runtime setup() cognitive complexity is only 12.
7	fastapi/param_functions.py	2,430	0.12	0	—	Pure declarative factory functions (Path, Query, Header, Cookie, Body, Form, File, Depends, Security) rich in @overload and Doc() metadata.
8	fastapi/dependencies/utils.py	1,294	0.12	0	—	Recursive signature analyzer (get_dependant, analyze_param) and async DAG solver (solve_dependencies); routing type cycle guarded by TYPE_CHECKING.

Representative Source Module Inventory (52 Modules, 19,837 LOC)

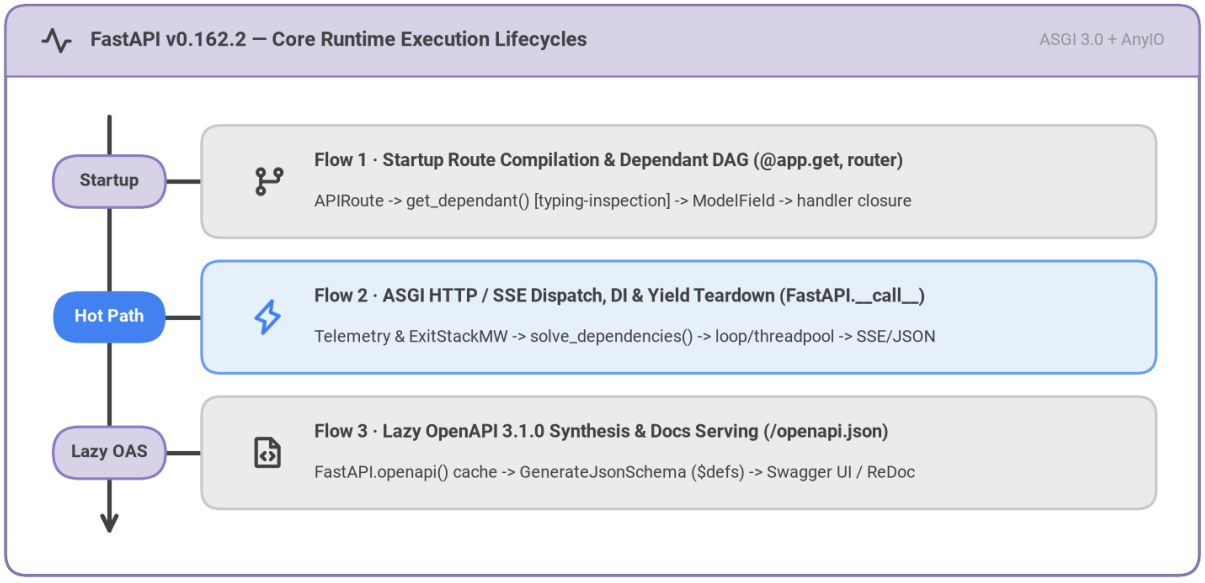
Module / Subpackage	Tier	LOC	Public Types	Key Callables & Architectural Role
fastapi/__init__.py	T5	22	*(20 PEP 484 X as X exports)*	Canonical package entrypoint exporting FastAPI, APIRouter, Depends, Query, Body, HTTPException, status, and UploadFile.
fastapi/applications.py	T5	4,118	FastAPI	Top-level ASGI application orchestrating middleware stack assembly, OpenTelemetry lifespan wrapping, OpenAPI caching, and route registration.

Module / Subpackage	Tier	LOC	Public Types	Key Callables & Architectural Role
`fastapi/param_functions.py`	T5	`2,430`	—	Developer-facing `Path`, `Query`, `Header`, `Cookie`, `Body`, `Form`, `File`, `Depends`, and `Security` factory functions.
`fastapi/responses.py` & `background.py`	T5	`126`	`ORJSONResponse`, `UJSONResponse`, `BackgroundTasks`	Optional high-throughput JSON response classes (`76` LOC) and telemetry-aware `BackgroundTasks` subclass (`50` LOC).
`fastapi/cli.py` & `__main__.py`	T5	`12`	—	`main()` CLI entrypoint delegating cleanly to `fastapi_cli.cli.main` (raises `RuntimeError` without `sys.exit` if `[standard]` extra is absent).
`fastapi/{requests,websockets,staticfiles,templating,testclient}.py`	T5	`8`	—	Five 1–3 LOC convenience re-export modules surfacing Starlette's `Request`, `WebSocket`, `StaticFiles`, `Jinja2Templates`, and `TestClient`.
`fastapi/routing.py`	T4	`5,726`	`APIRouter`, `APIRoute`, `APIWebSocketRoute`, `RouteContext`	Core route compiler (`get_request_handler`, `get_websocket_app`, `run_endpoint_function`, `serialize_response`, `iter_route_contexts`).
`fastapi/sse.py`	T4	`190`	`EventSourceResponse`, `ServerSentEvent`	Zero-internal-dependency native Server-Sent Events serializer (`format_sse_event`) and streaming response engine.
`fastapi/openapi/utils.py`	T4	`631`	—	`get_openapi`, `get_openapi_path`, `get_fields_from_routes`, `get_openapi_operation_metadata` — OpenAPI `3.1.0` specification synthesizer.
`fastapi/openapi/models.py` & `constants.py`	T4	`319`	`OpenAPI`, `PathItem`, `Operation`, `Schema`, `Components`, `OAuthFlows` *(+34)*	Strictly typed Pydantic v2 models representing the OpenAPI `3.1.0` & JSON Schema specification (`316` LOC) plus `REF_PREFIX` constants (`3` LOC).
`fastapi/openapi/docs.py`	T4	`341`	—	`get_swagger_ui_html`, `get_redoc_html`, `get_swagger_ui_oauth2_redirect_html` — interactive browser documentation renderers.
`fastapi/exception_handlers.py`	T4	`28`	—	`http_exception_handler`, `request_validation_exception_handler`, `websocket_request_validation_exception_handler`.
`fastapi/dependencies/models.py`	T3	`193`	`Dependant`	Immutable-style dataclass node representing a callable's typed parameters, sub-dependencies, security requirements, and cache key.
`fastapi/dependencies/utils.py`	T3	`1,294`	`ParamDetails`, `SolvedDependency`	`get_dependant`, `analyze_param`, `solve_dependencies`, `request_params_to_args`, `request_body_to_args` — static analyzer and runtime DI executor.

Module / Subpackage	Tier	LOC	Public Types	Key Callables & Architectural Role
`fastapi/security/{oauth2,http,api_key,open_id_connect_url,base,utils}.py`	T3	1,294	`OAuth2PasswordBearer`, `SecurityScopes`, `HTTPBearer`, `HTTPBasic`, `HTTPDigest`, `APIKeyHeader`, `OpenIdConnect`	Seven-file pluggable authentication & scope-enforcement suite integrated natively with `Depends()` / `Security()` (`secrets.compare_digest` in HTTP Basic/Digest).
`fastapi/telemetry/{_asgi,_api,_runtime,__init__}.py`	T2	869	`NativeTelemetry`, `ExceptionTelemetryMiddleware`, `TelemetryConfig`, `TelemetryData`	Four-file native OpenTelemetry subsystem (`get_telemetry_data`, `lifespan`) capturing ASGI HTTP/WebSocket spans and uncaught exceptions.
`fastapi/concurrency.py` & `middleware/*`	T2	55	`AsyncExitStackMiddleware`	`contextmanager_in_threadpool` (`31` LOC) plus `AsyncExitStackMiddleware` (`12` LOC) and 6 Starlette middleware re-exports (`12` LOC).
`fastapi/_compat/{v2,shared,__init__}.py`	T1	617	`ModelField`, `GenerateJsonSchema`	`evaluate_forwardref`, `get_definitions`, `create_body_model`, `lenient_issubclass`, `field_annotation_is_scalar` — Pydantic v2 adapter layer.
`fastapi/params.py`	T1	763	`Param`, `ParamTypes`, `Path`, `Query`, `Header`, `Cookie`, `Body`, `Form`, `File`, `Depends`, `Security`	Descriptor subclasses extending Pydantic `FieldInfo` with parameter location (`in_`), media types, and OpenAPI metadata.
`fastapi/encoders.py`	T1	333	`Color`, `PyExtraColor`	`jsonable_encoder`, `isoformat`, `decimal_encoder` — recursive Python-object-to-JSON-primitive serializer.
`fastapi/exceptions.py`	T1	201	`FastAPIError`, `HTTPException`, `RequestValidationError`, `ResponseValidationError`, `DependencyScopeError` *(+6)*	Structured framework, validation, dependency-scope, and WebSocket exception hierarchy.
`fastapi/{datastructures,utils,types,logger}.py`	T1	267	`UploadFile`, `DefaultPlaceholder`	`Default()` sentinel & `UploadFile` wrapper (`141` LOC), `create_model_field` / `deep_dict_update` (`114` LOC), type aliases (`10` LOC), and named `fastapi` logger (`2` LOC).

2.4 Entry Points, Runtime Execution Lifecycles, State & Concurrency Model

FastAPI separates **import-time signature compilation** from **request-time asynchronous execution** and **lazy first-hit OpenAPI synthesis**. Figure 2.3 traces these three lifecycles along a vertical execution roadmap.



FastAPI v0.162.2 Core Runtime Lifecycles: Startup Route Compilation, Per-Request DI Dispatch & Lazy OpenAPI Synthesis

---

Public Entry Points & Three Execution Lifecycles

FastAPI exposes four canonical entry points: `fastapi/__init__.py` (re-exporting 20 public symbols via explicit PEP 484 `X as X` aliases), `fastapi/applications.py:FastAPI`, `fastapi/routing.py:APIRouter`, and `fastapi/cli.py:main`.

1. **Flow 1 — Startup Route Compilation & `Dependant` DAG Construction**
   - **Trigger**: Decorator registration (`@app.get` , `@router.post`) or router composition (`app.include_router`) at module import / startup time.
   - **Execution Chain**: `FastAPI` / `APIRouter` instantiates an `APIRoute` (or `APIWebSocketRoute`) in `fastapi/routing.py` -> `APIRoute.__init__` invokes

`get_dependant()` and `analyze_param()` (`fastapi/dependencies/utils.py`), using `typing-inspection` to classify path, query, header, cookie, body, `Depends()`, and `Security()` parameters -> multi-field or scalar body parameters are compiled into Pydantic v2 `ModelField` / `TypeAdapter` validators (`fastapi/_compat/v2.py`) -> `APIRoute.get_route_handler()` pre-builds the asynchronous ASGI `request_response` closure via `get_request_handler()`, shifting reflection work out of the per-request hot path.

2. **Flow 2 — ASGI HTTP / SSE Request Dispatch, DI Resolution & `AsyncExitStack` Yield Teardown**
   - **Trigger**: Incoming ASGI HTTP or WebSocket `scope` delivered to `FastAPI.__call__(scope, receive, send)`.
   - **Execution Chain**: `NativeTelemetry` (when configured), `ServerErrorMiddleware`, user ASGI middlewares, `ExceptionMiddleware`, and `AsyncExitStackMiddleware` wrap the request -> Starlette's `Router` matches `APIRoute` and invokes its pre-compiled `get_request_handler()` closure -> JSON, form, or multipart request bodies are parsed and `solve_dependencies()` traverses the `Dependant` DAG using a per-request `dependency_cache` while pushing generator `yield` dependencies onto `AsyncExitStack` -> `run_endpoint_function()` awaits `async def` callables directly on the event loop or offloads synchronous `def` callables via `run_in_threadpool` -> `serialize_response()` validates/filters return payloads through Pydantic v2 `ModelField` (or streams `EventSourceResponse` / JSONL chunks), transmits ASGI response frames, unwinds request-scoped `yield` dependencies inside `AsyncExitStackMiddleware`, and executes `BackgroundTasks`.
3. **Flow 3 — Lazy OpenAPI 3.1.0 Schema Synthesis & Interactive Docs Serving**
   - **Trigger**: First `GET /openapi.json` request (or explicit `app.openapi()` invocation) alongside `GET /docs` or `GET /redoc`.
   - **Execution Chain**: `FastAPI.openapi()` checks `self.openapi_schema`; on first-access cache miss, it invokes `get_openapi()` (`fastapi/openapi/utils.py`) -> `get_fields_from_routes()` gathers every input/output `ModelField` across routes and `Dependant` trees, generating deduplicated JSON Schema `$defs` via `GenerateJsonSchema` (`fastapi/_compat/v2.py`) -> `get_openapi_path()` synthesizes operation metadata, security schemes, `requestBody`, callbacks, and status-code responses into an OpenAPI `3.1.0` dictionary cached on `app.openapi_schema` -> `fastapi/openapi/docs.py` serves Swagger UI (`/docs`) or ReDoc (`/redoc`) HTML bound to `openapi_url`.

State Management, Ephemeral ASGI Scope Storage & Concurrency Model

- **Stateless Process Footprint & In-Memory Metadata**: `FastAPI` maintains zero database or filesystem state. Instance attributes hold only immutable-after-startup route lists (`APIRoute`), `dependency_overrides` mappings for test injection, middleware descriptors, and the lazily populated `self.openapi_schema` dictionary.
- **Two-Stage Ephemeral ASGI Scope Teardown (`fastapi_function_astack` vs. `fastapi_inner_astack`)**: During each request, FastAPI attaches two distinct `contextlib.AsyncExitStack` instances to the mutable ASGI `scope` dictionary alongside a per-request `dependency_cache` keyed by `(call, tuple(security_scopes))`:
  - `scope["fastapi_function_astack"]` (`scope="function"` yield dependencies): Entered before endpoint execution and **exited immediately after endpoint return / response serialization**, allowing `yield` cleanup blocks to raise or transform HTTP exceptions before headers are committed.
  - `scope["fastapi_inner_astack"]` (`scope="request"` yield dependencies, managed by `AsyncExitStackMiddleware`): Remains open across streaming bodies (`StreamingResponse`, `EventSourceResponse`) and **exits only after final ASGI response frames complete**, guaranteeing database sessions or streaming cursors are never closed mid-stream.
- **AnyIO Concurrency & Structured `422` Validation Model**: Backed by Starlette and AnyIO, `async def` endpoints and dependencies run directly on the active `asyncio` or `trio` event loop, while synchronous `def` endpoints and sync generator `yield` dependencies (`contextmanager_in_threadpool` in `fastapi/concurrency.py`) are automatically dispatched to AnyIO worker threads to prevent event-loop stalls. Input or contract violations raise `RequestValidationError`, `WebSocketRequestValidationError`, or `ResponseValidationError` (`fastapi/exceptions.py`), which `fastapi/exception_handlers.py` converts into deterministic HTTP `422` JSON error payloads.

3. Recommended Remediation Roadmap

FastAPI `v0.162.2` enters remediation planning from an exceptional **93/100 (Grade S)** baseline with **zero** `P0` **urgent blockers**, **zero** `FAIL` **compliance rules**, and **zero** `CRITICAL` **or** `HIGH` **source-code findings** across all 52 framework modules and 3,216 passing hermetic tests. Rather than emergency defect patching, the 6-item engineering roadmap focuses on **2** `P1` **high-leverage structural and lockfile refinements** followed by **4** `P2` **architectural, repository governance, test-assertion, and typing polish items**.

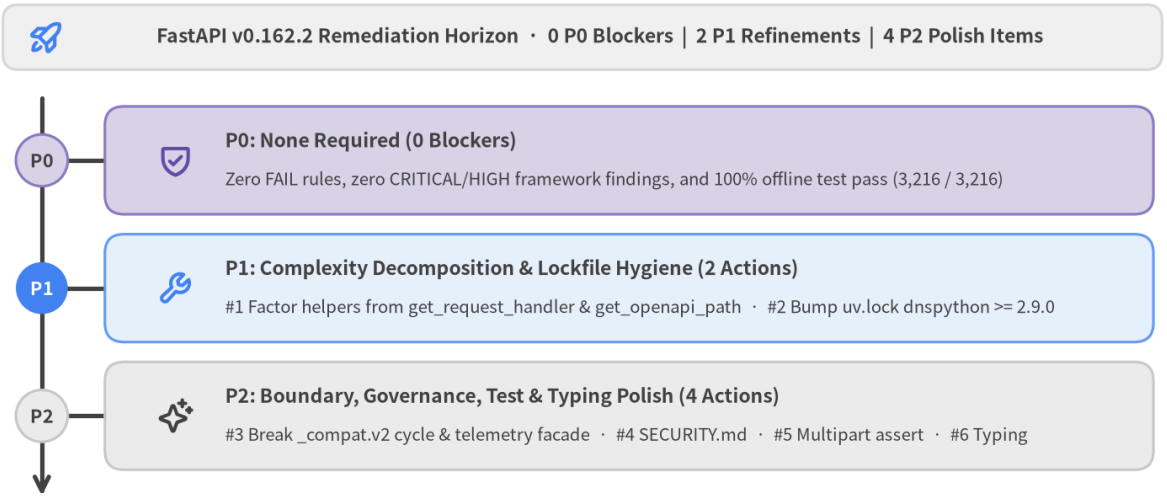


Figure 3.1 — Phased Engineering Refinement Roadmap for FastAPI v0.162.2 (0 P0 Urgent Blockers · 2 P1 High-Leverage · 4 P2 Polish)

3.1 Action Plan Traceability Matrix (`P0 -> P1 -> P2`)

Every remediation candidate identified by PNI501 maps deterministically to concrete file locations, expected engineering outcomes, and underlying compliance rules or vulnerability advisories. Because FastAPI `v0.162.2` ships with zero `CRITICAL` / `HIGH` framework defects and flexible `>=` SemVer bounds on its 6 runtime dependencies, the action plan begins at `P1` and preserves exact priority ordering (`1..6`):

Remediation Action Plan Traceability Matrix (0 P0 Urgent · 2 P1 High-Leverage · 4 P2 Polish)

Priority	Action Title	Target Files	Expected Impact	Related Rules
[P1] #1 code_quality	Decompose request handler & OpenAPI path synthesis routines	fastapi/routing.py fastapi/openapi/utils.py fastapi/telemetry/_asgi.py	Cuts cognitive hotspots toward <=25 while keeping pre-computation speed	common/maintainability python/code_quality
[P1] #2 security	Bump dnspython>=2.9.0 in uv.lock & refresh test harness base image	uv.lock .pni/dockerfiles/root/ Dockerfile	Eliminates single HIGH uv.lock advisory & trims harness OS CVE noise	common/security_vulnerability CVE-2026-114605
[P2] #3 architecture	Break _compat.v2 / params / datastructures cycle & re-export telemetry facade	fastapi/_compat/v2.py fastapi/params.py, datastr. fastapi/telemetry/__init__.py	Zero runtime circular dependencies across 52 modules & clean boundary	common/architecture
[P2] #4 security	Add showcase SECURITY.md & GitHub Actions Dependabot configuration	README.md pyproject.toml	Satisfies OpenSSF checks while routing advisories to upstream FastAPI	common/supply_chain
[P2] #5 testing	Add explicit warning assertion in test_old_multipart_installed	tests/test_multipart_ installation.py	Prevents silent pass if multipart version warning behavior regresses	python/testing
[P2] #6 typing	Replace mutable dict default in ModelField.validate & scope type: ignore	fastapi/_compat/v2.py fastapi/params.py fastapi/dependencies/utils.py	Eliminates B006 default hazard & lifts callable typing above 99%	python/code_quality python/typing

Figure 3.2 — Action Plan Traceability Matrix Mapping Priorities, Target Files, Expected Impact, and Audit Rules

Detailed Pre-Sorted Remediation Backlog (`1..6`)

#	Priority & Domain	Action Title & Implementation Guidance	Target Files	Expected Engineering
1	`[P1]` `code_quality`	**Decompose high-complexity request handler and OpenAPI path synthesis routines** Factor body extraction, dependency execution, and response serialization helpers out of `get_request_handler` (`cognitive=159`) in `fastapi/routing.py` and split response/callback schema builders out of `get_openapi_path` (`cognitive=109`) in `fastapi/openapi/utils.py` (alongside span-attribute helpers in `fastapi/telemetry/_asgi.py`).	`fastapi/routing.py` , `fastapi/openapi/utils.py` , `fastapi/telemetry/_asgi.py`	Reduces cognitive cor hotspots toward `<= 2` preserving FastAPI's r registration pre-compu speed.

#	Priority & Domain	Action Title & Implementation Guidance	Target Files	Expected Engineering
2	`[P1]` `security`	**Bump transitive `dnspython>=2.9.0` in `uv.lock` and refresh test harness base image** Upgrade locked transitive `dnspython` from `2.8.0` to `>=2.9.0` via `uv lock --upgrade-package dnspython` (closing `CVE-2026-114605`) and run `apt-get update && apt-get upgrade -y` or switch to `trixie-slim` in `.pni/dockerfiles/root/Dockerfile` for hermetic CI runs.	`uv.lock` , `.pni/dockerfiles/root/Dockerfile`	Eliminates the single transitive `uv.lock` ad and reduces test-harn base-image CVE noise
3	`[P2]` `architecture`	**Break runtime import cycle across `_compat.v2` , `params` , and `datastructures` and re-export telemetry facade** Move runtime `UploadFile` /sentinel checks or guard under `TYPE_CHECKING` to eliminate the `_compat.v2 <-> params -> datastructures` cycle, and expose internal telemetry entrypoints via `fastapi/telemetry/__init__.py` .	`fastapi/_compat/v2.py` , `fastapi/params.py` , `fastapi/datastructures.py` , `fastapi/telemetry/__init__.py`	Achieves zero runtime circular dependencies all 52 modules and cle subpackage boundary encapsulation.
4	`[P2]` `security`	**Add showcase `SECURITY.md` and GitHub Actions Dependabot configuration** Add a concise `SECURITY.md` redirecting FastAPI vulnerability disclosures to the official upstream FastAPI Security Advisories page, and add `.github/dependabot.yml` for CI workflow maintenance.	`README.md` , `pyproject.toml`	Satisfies OpenSSF Sc repository governance while clearly routing framework security disclosures upstream.
5	`[P2]` `testing`	**Add explicit warning assertion in `test_old_multipart_installed`** Capture `as w` on `warnings.catch_warnings(record=True)` in `tests/test_multipart_installation.py:141` and assert expected warning count/message.	`tests/test_multipart_installation.py`	Eliminates silent asse free test pass if multip version warning behav regresses.
6	`[P2]` `typing`	**Replace mutable `dict` default in `ModelField.validate` and scope remaining `type: ignore` comments** Replace `values: dict[str, Any] = {}` with `values: Mapping[str, Any]	None = None in fastapi/_compat/v2.py:176 , add -> None on Param.__init__ subclasses, and attach explicit [error-code] brackets to the 5 unscoped # type: ignore` comments.	`fastapi/_compat/v2.p` `fastapi/params.py` , `fastapi/dependencies.py`

3.2 Engineering Rationale & Execution Guidance

Why Zero `P0` Urgent Actions Are Required

A `P0` designation in PNI501 is reserved for build failures, failing hermetic test gates, exploitable CWE execution sinks, leaked secrets, or `CRITICAL` / `HIGH` vulnerabilities inside shipped runtime dependencies. FastAPI `v0.162.2` clears every one of these gates cleanly: all **6 core runtime dependencies** (`starlette` , `pydantic` , `typing-extensions` , `typing-inspection` , `annotated-doc` , `opentelemetry-api`) report **0 CVEs**, static SAST inspection across all 52 source modules found **0 dangerous sinks** and **0 hardcoded secrets**, and containerized offline testing (`--network=none` , UID `10001`) passed **3,216 of 3,216** executed tests (`96.4%` line coverage, `91.0%` branch coverage).

Preserving Hot-Path Performance During `P1` Complexity Refactoring (`#1`)

FastAPI's signature performance advantage comes from shifting parameter introspection and Pydantic v2 `TypeAdapter` construction to **startup route registration**, emitting a specialized async closure inside `get_request_handler` (`fastapi/routing.py:397` , `cognitive=159`) and lazy-caching OpenAPI 3.1.0 synthesis in `get_openapi_path` (`fastapi/openapi/utils.py:303` , `cognitive=109`). When executing Action `#1` , maintainers should preserve startup pre-computation by extracting pure module-level helper functions (`_parse_request_body` , `_execute_solved_endpoint` , `_build_validated_response`) that receive pre-bound route metadata rather than re-inspecting signatures per request. This keeps per-request ASGI overhead identical while making each stage independently unit-testable and readable.

Contextualizing Lockfile & Test-Harness Advisories (`#2`)

Action `#2` addresses `CVE-2026-114605` (`CVSS 7.5` , `dnspython@2.8.0 -> 2.9.0` in `uv.lock`). Because `pyproject.toml` does not pin `dnspython` in FastAPI's core wheel metadata—it enters `uv.lock` solely via the optional `email-validator` extra—downstream applications installing `fastapi` automatically resolve `dnspython>=2.9.0` . Running `uv lock --upgrade-package dnspython` and refreshing the ephemeral Debian test-runner container

image (`.pni/dockerfiles/root/Dockerfile`) cleanly eliminates automated SCA alert noise in CI pipelines.

Architectural, Governance, Test, and Typing Polish (`#3` – `#6`)

The four `P2` items tighten already-strong internal invariants without altering public API behavior:

- **Subpackage Encapsulation (`#3`)**: Guarding type-only `UploadFile` / `FieldInfo` references under `if TYPE_CHECKING:` breaks the lone Tier-1 runtime cycle (`fastapi._compat.v2 <-> fastapi.params -> fastapi.datastructures`), while re-exporting public symbols in `fastapi/telemetry/__init__.py` eliminates private cross-module imports (`_asgi` , `_runtime` , `_api`).
- **Showcase Repository Governance (`#4`)**: Adding a lightweight `SECURITY.md` (pointing vulnerability reports to upstream FastAPI Security Advisories) and `.github/dependabot.yml` aligns standalone repository snapshots with OpenSSF Scorecard expectations.
- **Test & Type Completeness (`#5` , `#6`)**: Binding `warnings.catch_warnings(record=True) as recorded` in `tests/test_multipart_installation.py:141` guarantees warning regressions cannot pass silently, while normalizing `values: Mapping[str, Any] | None = None` in `ModelField.validate` (`fastapi/_compat/v2.py:176`), adding `-> None` return annotations on descriptor `__init__` methods, and scoping the 5 `# type: ignore[...]` comments push callable type coverage past `99%` .

4. Audit Standards, Rule Compliance & Findings

PNI501 evaluated FastAPI `v0.162.2` across **6 deterministic audit steps** and **15 engineering compliance rules** spanning static AST maintainability, supply-chain security, PEP 621/561 packaging, library API hygiene, test determinism, and hermetic `--network=none` container execution. Every audit step achieved Grade `A` or `S` (`89/100` to `97/100`), yielding **10** `PASS` **rules**, **5 advisory** `WARN` **rules**, **0** `FAIL` **rules**, and **8 consolidated** `MEDIUM` / `LOW` **location findings** (`0 CRITICAL` , `0 HIGH`).

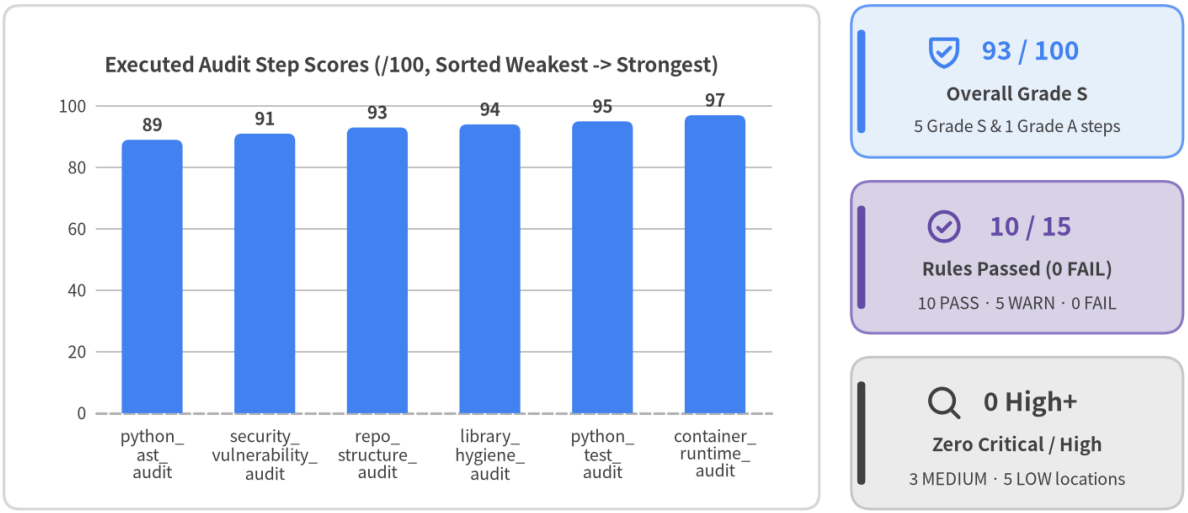


Figure 4.1 — Executed PNI501 Audit Step Score Comparison (Weakest-First Ascending: 89 to 97) & Compliance Posture

4.1 Executed Audit Step Score Summary (`89 -> 97`)

All six audit primitives completed deterministically without execution errors. Sorted ascending from lowest score to highest score to spotlight improvement headroom first, FastAPI `v0.162.2` posts five **Grade S** (`>= 90`) ratings and one high **Grade A** (`89/100`) rating:

Audit Step ID	Score & Grade	Rule Tally (`[pass/warn/fail]`)	Key Diagnostic Synthesis
`python_ast_audit`	**89 / 100 (A)**	`[pass=4, warn=3, fail=0]`	Across 52 Python modules (`19,837` LOC, `121` classes, `438` functions), FastAPI `v0.162.2` demonstrates exceptional security and typing discipline: `94.5%` callable type safety coverage, packaged PEP 561 `py.typed` , PEP 727 `Annotated[..., Doc()]` inline parameter documentation (`annotated_doc`), zero wildcard imports, zero empty or mismatched docstrings, zero hardcoded secrets, and zero dangerous CWE execution sinks. Structural complexity concentrates intentionally in five core runtime engines— `get_request_handler` (`cognitive=159`), `get_openapi_path` (`cognitive=109`), `NativeTelemetry.__call__` (`cognitive=85`), `analyze_param` (`cognitive=64`), and `solve_dependencies` (`cognitive=33`)—alongside a single runtime import cycle (`fastapi._compat.v2 <-> fastapi.params -> fastapi.datastructures`).
`security_vulnerability_audit`	**91 / 100 (S)**	`[pass=0, warn=2, fail=0]`	Core runtime dependencies (`starlette` , `pydantic` , `typing-extensions` , `typing-inspection` , `annotated-doc` , `opentelemetry-api`) contain **zero** `CRITICAL` **or** `HIGH` **CVEs**, zero hardcoded or committed secrets, zero container IaC misconfigurations, and zero dangerous CWE execution sinks across all 52 source modules. Trivy SCA identified 7 transitive lockfile advisories in `uv.lock` (`0 CRITICAL` , `1 HIGH` : `CVE-2026-114605` in optional `email-validator` 's transitive `dnspython@2.8.0 -> 2.9.0` , `5 MEDIUM` , `1 LOW`)— none affecting downstream consumers due to flexible `>=` SemVer bounds—while the `398` OS package advisories originate solely from the ephemeral Debian `12.13` test harness base image.
`repo_structure_audit`	**93 / 100 (S)**	`[pass=4, warn=1, fail=0]`	Exhibits exemplary PEP 621 packaging and dependency hygiene: 6 lean runtime dependencies with explicit SemVer lower bounds, zero unpinned specs, isolated PEP 735 `dependency-groups` , a deterministic `uv.lock` lockfile, and a packaged PEP 561 `py.typed` marker across 52 source modules and 273 core test modules. Only repository-level OpenSSF governance files (`SECURITY.md` and Dependabot config) are absent in this standalone showcase snapshot.

Audit Step ID	Score & Grade	Rule Tally (`[pass/warn/fail]`)	Key Diagnostic Synthesis
`library_hygiene_audit`	**94 / 100 (S)**	`[pass=2, warn=1, fail=0]`	Exemplifies gold-standard Python library hygiene: explicit PEP 484 `X as X` re-exports in `fastapi/__init__.py` across 20 canonical symbols, zero import-time I/O or network side effects, zero `sys.exit()` calls inside library code (`fastapi/cli.py` raises `RuntimeError` cleanly when `fastapi-cli` is absent), named `logging.getLogger('fastapi')` without `basicConfig` hijacking (`fastapi/logger.py`), and flexible `>=` lower-bound SemVer constraints across all 6 core dependencies.
`python_test_audit`	**95 / 100 (S)**	`[pass=1, warn=0, fail=0]`	Core framework test suite spans 273 test files and `1,835` test functions with `100.0%` static module test coverage across all 52 source modules (`5.25x` test-to-source file ratio). Strict `pytest` configuration (`--strict-config` , `--strict-markers` , `strict_xfail=true` , `filterwarnings=['error']` , `timeout=20`) enforces high determinism and warning-free execution, with only 2 assertion-free helper/warning test functions identified.
`container_runtime_audit`	**97 / 100 (S)**	`[pass=2, warn=0, fail=0]`	Hermetic containerized runtime verification (`--network=none` , non-root UID `10001` , Python `3.12`) passed cleanly across all gates: clean frozen `uv.lock` container build (`0.79s` cached), `100%` offline test pass across `3,228` collected items (`3,216` passed, `0` failed, `0` errors, `12` skipped in `265.57s`), `96.4%` measured line coverage (`4,628 / 4,801` statements) and `91.0%` branch coverage across `fastapi/` , and a clean zero-side-effect library import smoke check.

4.2 Full 15-Rule Compliance Matrix (`FAIL -> WARN -> PASS`)

Across all 15 evaluated PNI501 engineering standards, FastAPI `v0.162.2` records `0 FAIL` , `5 WARN` , and `10 PASS` checks. The table below preserves the exact pre-sorted compliance order (`WARN` advisories first, followed by `PASS` standards):

Status	Rule ID	Evaluated By	Diagnostic Evidence & Verification Summary
[WARN]	common/architecture	python_ast_audit	1 runtime circular dependency (fastapi._compat.v2 -> fastapi.params -> fastapi.datastructures -> fastapi._compat.v2) and 4 private cross-domain imports from fastapi.applications / routing / background into fastapi.telemetry._asgi / _runtime / _api ; 2 additional cycles are cleanly guarded under TYPE_CHECKING .
[WARN]	common/maintainability	python_ast_audit	get_request_handler (fastapi/routing.py:397 , cognitive=159 , 408 LOC), get_openapi_path (fastapi/openapi/utils.py:303 , cognitive=109 , 230 LOC), NativeTelemetry.__call__ (fastapi/telemetry/_asgi.py:228 , cognitive=85), analyze_param (fastapi/dependencies/utils.py:409 , cognitive=64), and 52 wide ergonomic OpenAPI decorator signatures (FastAPI.__init__ params= 37 , get / post / put / delete params= 23).
[WARN]	common/security_vulnerability	security_vulnerability_audit	dangerous_sinks=[] across all 52 framework modules; built-in security primitives (OAuth2 , HTTPBearer , HTTPBasic / Digest using secrets.compare_digest , APIKey , OpenIdConnect) enforce structured credential extraction. SEC-01 app_vuln_counts={critical:0, high:1, medium:5, low:1, total:7} in transitive dev/optional uv.lock (CVE-2026-114605 dnspython@2.8.0 -> 2.9.0) with 0 CVEs in core runtime dependencies; os_vuln_counts=398 in ephemeral Debian 12.13 test harness image; SEC-02 misconfigurations=[] ; SEC-03 secrets=[] ; SEC-04 dangerous_sinks=[] ; SEC-05 library security extractors enforce validation.
[WARN]	common/supply_chain	repo_structure_audit	lockfiles_found=['uv.lock'] , has_license=true (LICENSE + LICENSE-FASTAPI), ci_supply_chain_issues=[] , 0 unpinned dependencies, and non-root UID 10001 container harness (0 IaC misconfigurations); has_security_policy=false and has_dependency_automation=false because standalone showcase snapshot omits SECURITY.md and Dependabot config.
[WARN]	python/code_quality	python_ast_audit	empty_docstrings=[] , doc_signature_mismatches=[] , wildcard_imports=[] ; conventional AST docstring_coverage_pct=25.2% because FastAPI adopts PEP 727 Annotated[..., Doc()] inline parameter documentation rather than traditional docstring blocks; 1 mutable default (fastapi/_compat/v2.py:176) and 3 silent except-pass blocks in fastapi/routing.py:640,1880,2097 .
[PASS]	common/documentation	repo_structure_audit	has_readme=true (59 lines, 6 structured sections covering Purpose, Layout, Reproducible uvx Commands, and Licensing) + packaged fastapi/py.typed marker; PEP 727 Annotated[..., Doc()] inline documentation across all public classes/functions (FastAPI , APIRouter , Path , Query , Body , Depends , Security), zero doc_signature_mismatches , and zero empty_docstrings .
[PASS]	common/git_workflow	repo_structure_audit	has_gitignore=true (.venv , __pycache__ , .drawlib/ , .pni/cache/ excluded); clean linear main branch with zero conflict markers.
[PASS]	common/library_guidelines	library_hygiene_audit	fastapi/__init__.py uses PEP 484 explicit from ... import X as X re-exports (20 public symbols), fastapi/py.typed is packaged, fastapi/logger.py uses logging.getLogger('fastapi') with zero root-logger hijacking, zero sys.exit() calls exist in fastapi/ , and all 6 core dependencies in pyproject.toml use flexible >= floors without upper-bound caps.
[PASS]	common/production_readiness	container_runtime_audit	missing_io_timeouts=0 , native OpenTelemetry span/exception instrumentation in fastapi/telemetry/_asgi.py (94.9% line, 93.8% branch coverage), deterministic AsyncExitStack teardown, and structured validation exception handlers.

Status	Rule ID	Evaluated By	Diagnostic Evidence & Verification Summary
`[PASS]`	`common/runtime_verification`	`container_runtime_audit`	`RV-01` `harness_build=passed` (`exit_code=0`, `.pni/dockerfiles/root/Dockerfile`); `RV-02` `test_execution=passed` under `--network=none` (`3,216` passed, `0` failed, `0` errors, `12` skipped); `RV-03` measured `line_coverage_pct=96.4%` (`4,628/4,801`) and `branch_coverage_pct=91.0%`; `RV-04` library `smoke_check=passed` (`'Imported fastapi cleanly'`).
`[PASS]`	`common/security`	`repo_structure_audit`	`committed_secret_files=[]`, `container_hygiene_issues=[]`, non-root UID `10001` configured in `.pni/dockerfiles/root/Dockerfile`; `hardcoded_secrets=[]`, `dangerous_sinks=[]`, `missing_io_timeouts=[]`, `global_state_usages=0`; 1 minor mutable default dict `values={}` in `fastapi/_compat/v2.py:176` (`ModelField.validate`, never mutated in-place).
`[PASS]`	`python/dependencies`	`repo_structure_audit`	`pyproject.toml` declares 6 explicit SemVer-floored runtime dependencies (`starlette>=0.46.0`, `pydantic>=2.9.0`, `typing-extensions>=4.8.0`, `typing-inspection>=0.4.2`, `annotated-doc>=0.0.2`, `opentelemetry-api>=1.44.0`), `unpinned_dependencies=[]`, isolated `[dependency-groups]` (`dev`, `tests`, `docs`), and committed `uv.lock`.
`[PASS]`	`python/modern_stack`	`python_ast_audit`	Modern Python `>=3.10..3.14` stack using Pydantic v2 `TypeAdapter`, `typing-inspection`, `annotated-doc` (PEP 727), OpenTelemetry API, and `uv.lock`. The 38 `isort` group notices stem from FastAPI's explicit `[tool.ruff.lint.isort]` `known-third-party=['fastapi','pydantic','starlette']` convention; 6 `os.path` calls remain in `fastapi/routing.py` static SPA path resolution.
`[PASS]`	`python/testing`	`python_test_audit`	273 core test files, `1,835` test functions, `module_test_coverage_pct=100.0%`, strict `pytest` settings (`strict_xfail=true`, `filterwarnings=['error']`, `timeout='20'`), and `inline_snapshot` OpenAPI 3.1 schema verification; only 2 assertion-free test functions (`tests/test_multipart_installation.py:141` and `tests/test_webhooks_security.py:34`).
`[PASS]`	`python/typing`	`python_ast_audit`	`type_safety_coverage_pct=94.5%`, `has_type_marker=true` (`fastapi/py.typed`), strict `mypy` + `ty` configuration in `pyproject.toml`; 29 missing annotations are almost exclusively `__init__ -> None` return omissions in `fastapi/params.py` and `datastructures.py`, plus 5 unscoped `# type: ignore` comments.

4.3 Consolidated Location Findings Index (`CRITICAL -> INFO`)

Deduplicating raw tool observations by exact `file:line` yields **8 unique finding locations** (`0 CRITICAL`, `0 HIGH`, `3 MEDIUM`, `5 LOW`, `0 INFO`). Each row below maps a single source or manifest coordinate to its triggered rule ID, diagnostic evidence, and concrete remediation:



Consolidated Location Findings Matrix (8 Unique file:line Coordinates · 3 MEDIUM · 5 LOW)

Severity	Location (file:line)	Rule ID	Diagnostic Evidence	Recommended Fix
[MEDIUM] #1	fastapi/openapi/utils.py:303	common/maintainability	length=230 LOC, branches=29, cognitive=109	Factor response-schema & callback builders into helpers
[MEDIUM] #2	fastapi/routing.py:397	common/maintainability	length=408 LOC, branches=49, cognitive=159 (closure=157)	Extract body, DI & response stages; keep closure cache
[MEDIUM] #3	uv.lock:1	common/security_vulnerability	CVE-2026-114605 (HIGH 7.5): dnspython 2.8.0 -> 2.9.0	Run uv lock --upgrade-package dnspython to >=2.9.0
[LOW] #4	fastapi/_compat/v2.py:1	common/architecture	1 cycle: _compat.v2 <-> params -> datastructures	Guard UploadFile/FieldInfo under TYPE_CHECKING
[LOW] #5	fastapi/_compat/v2.py:176	python/code_quality	Mutable default argument 'values={}' (B006)	Default values=None and normalize inside method
[LOW] #6	fastapi/params.py:30	python/typing	29 missing -> None returns & 5 unscoped type: ignore	Add -> None on __init__ & scope # type: ignore[code]
[LOW] #7	pyproject.toml:1	common/supply_chain	has_security_policy=false, dependency_automation=false	Add upstream SECURITY.md and Dependabot config
[LOW] #8	tests/test_multipart_installation.py:141	python/testing	catch_warnings(record=True) has no warning assert	Bind as recorded and assert expected warning explicitly

Figure 4.2 — Consolidated Location Findings Overview (8 Unique Locations: 0 CRITICAL · 0 HIGH · 3 MEDIUM · 5 LOW)

Detailed Consolidated Location Findings (`1..8`)

#	Severity	Location (`file:line`)	Rule ID	Diagnostic Evidence	Concrete Fix Recommendation
1	`[MEDIUM]`	`fastapi/openapi/utils.py:303`	`common/maintainability`	`length=230 lines, branches=29, cognitive=109`	Factor response-schema assembly and callback operation generation into composable helper functions.
2	`[MEDIUM]`	`fastapi/routing.py:397`	`common/maintainability`	`length=408 lines, branches=49, cognitive=159 (inner app closure at line 428: cognitive=157)`	Extract dedicated helper stages (`_parse_request_body` , `_execute_solved_endpoint` , `_build_validated_response`) while preserving closure pre-computation for hot-path performance.
3	`[MEDIUM]`	`uv.lock:1`	`common/security_vulnerability`	`CVE-2026-114605 (HIGH, CVSS 7.5): dnspython 2.8.0 in uv.lock (fixed in 2.9.0; transitive dependency of optional email-validator)`	Run `uv lock --upgrade-package dnspython` in upstream maintenance cycles to bump locked `dnspython` from `2.8.0` to `>=2.9.0` .

#	Severity	Location (`file:line`)	Rule ID	Diagnostic Evidence	Concrete Fix Recommendation
4	`[LOW]`	`fastapi/_compat/v2.py:1`	`common/architecture`	`circular_dependencies=[['fastapi._compat.v2', 'fastapi.params', 'fastapi.datastructures', 'fastapi._compat.v2']]`	Guard type-only `UploadFile` / `FieldInfo` cross-references under `TYPE_CHECKING` or extract shared sentinel types into `fastapi.types` .
5	`[LOW]`	`fastapi/_compat/v2.py:176`	`python/code_quality`	`Mutable default argument 'values={}' (B006)`	Default `values` to `None` and normalize to `{}` or `MappingProxyType` inside the method body to prevent accidental shared-dict mutation.
6	`[LOW]`	`fastapi/params.py:30`	`python/typing`	`29 missing annotations (primarily __init__ -> None return annotations in fastapi/params.py and fastapi/datastructures.py) and 5 unscoped # type: ignore comments`	Add explicit `-> None` return annotations on descriptor `__init__` methods and scope remaining `# type: ignore[code]` directives.
7	`[LOW]`	`pyproject.toml:1`	`common/supply_chain`	`has_security_policy=false, has_dependency_automation=false`	If operating this showcase repository continuously, add a `SECURITY.md` pointing upstream FastAPI security reports to `github.com/fastapi/fastapi/security` and configure Dependabot for GitHub Actions.
8	`[LOW]`	`tests/test_multipart_installation.py:141`	`python/testing`	`with warnings.catch_warnings(record=True): ... contains no assert on captured warnings`	Bind `with warnings.catch_warnings(record=True) as recorded:` and assert the expected warning or clean registration behavior explicitly.

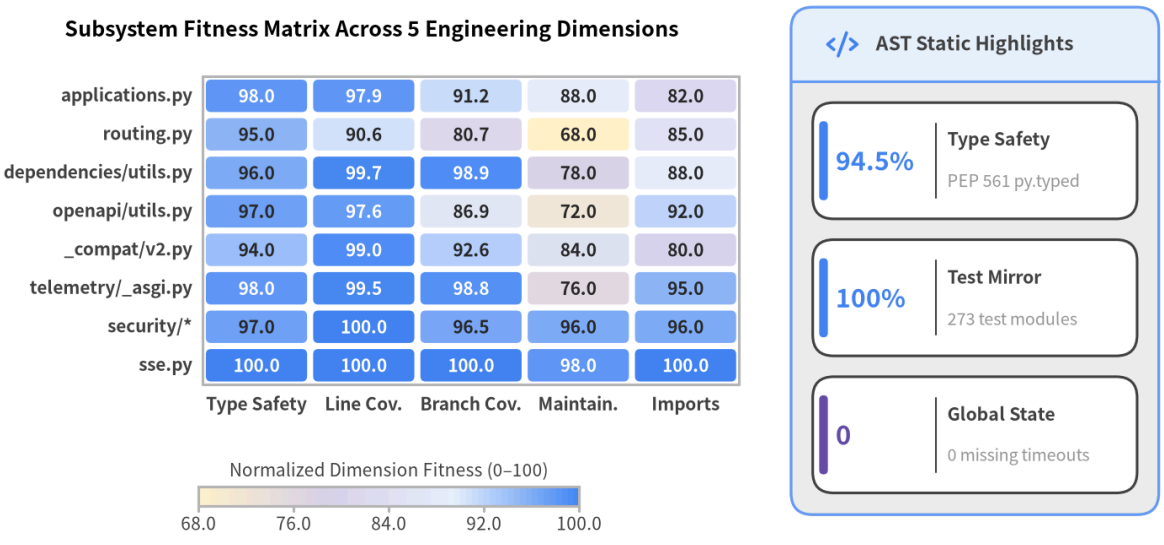
4.4 Configured Scope & Accepted Design Waivers (`pni.toml`)

PNI501 explicitly tracks project-level `[[opt_out]]` waivers in `pni.toml` so intentional infrastructure choices remain transparent and auditable rather than silently suppressed. Exactly **1 design waiver** is configured for this diagnostic run:

Waiver Rule ID	Configured Scope	Matched Findings	Engineering Justification
`DS-0026`	`[.pni/dockerfiles/**]`	`0`	Hermetic batch test container exits upon `pytest` completion; daemon `HEALTHCHECK` is not applicable.

5. Deep Dive: Code Quality, Testing & Runtime

FastAPI `v0.162.2` pairs expressive developer ergonomics with rigorous runtime engineering across `52` Python modules (`19,837` LOC, `121` classes, `438` functions). Static AST inspection confirms `94.5%` callable type-safety coverage, `0` global state usages, `0` wildcard imports, and `0` missing I/O timeouts, while hermetic `--network=none` container execution verifies `3,216` passing tests (`0` failures), `96.4%` line coverage (`4,628 / 4,801` statements), and `91.0%` branch coverage.



FastAPI v0.162.2 Subsystem × Engineering Quality Dimension Fitness Matrix (0–100 Scale)

5.1 Codebase Scale, Static Quality & Python Idiom Alignment

Deterministic Python AST analysis (`python_ast_audit`: **89/100, Grade A**) inspected all `52` framework modules (`19,837` lines of code, `121` classes, and `438` functions/methods)

alongside `273` core framework test files (`1,835` static test functions; `5.25x` test-to-source module ratio).

Static Quality & Coverage Metric	Measured Value	Diagnostic Interpretation
Source Modules (`total_files`)	`52`	Clean 5-tier package decomposition under `fastapi/`
Source Volume (`total_loc`)	`19,837` LOC	Includes rich inline PEP 727 `Doc()` reference prose on public APIs
Declared Classes & Callables	`121` classes / `438` functions	Balanced mix of Pydantic/OpenAPI models, descriptors, and ASGI compilers
Type Safety Coverage (`type_safety_coverage_pct`)	`94.5%`	Packaged `fastapi/py.typed` marker + strict `mypy` (`pydantic.mypy`) & `ty`
Static Module Test Mirror (`module_test_coverage_pct`)	`100.0%`	Every source module is exercised across `273` test files (`1,835` test functions)
Standard `isort` Hygiene (`import_hygiene_coverage_pct`)	`57.7%`	Reflects custom Ruff `known-third-party` grouping & PEP 484 `__init__.py` re-exports
Traditional AST Docstrings (`docstring_coverage_pct`)	`25.2%`	Superseded by PEP 727 `Annotated[..., Doc()]` inline parameter metadata (`0` drift)

Three architectural conventions explain the raw static AST percentages with high precision:

1. **Strict PEP 561 Typing Discipline (`94.5%` Callable Coverage)**: FastAPI ships an explicit `fastapi/py.typed` marker and enforces both `mypy` (`strict = true`) and Astral `ty` in `pyproject.toml`. Of the `29` unannotated callables flagged by raw AST inspection, almost all are benign `__init__` methods omitting `-> None` return annotations in `fastapi/params.py` and `fastapi/datastructures.py` (plus `Path.in_` at `fastapi/params.py:143`), accompanied by only `5` unscoped `# type: ignore` directives (`fastapi/cli.py:5` , `fastapi/datastructures.py:181` , `fastapi/dependencies/utils.py:671,688` , and `fastapi/security/http.py:204`). Deliberate `Any` annotations (`404` occurrences) are cleanly confined to runtime type-introspection boundaries (`fastapi/_compat/` , `fastapi/dependencies/` , and `fastapi/encoders.py`).
2. **Intentional Ruff `isort` Grouping (`57.7%` Standard `isort` Match)**: Standard `isort` heuristics report `38` import-ordering notices because FastAPI intentionally configures `[tool.ruff.lint.isort] known-third-party = ["fastapi", "pydantic", "starlette"]` in `pyproject.toml` . Treating `fastapi` imports identically across framework internals and `docs_src/` tutorial modules keeps contributor formatting uniform. Similarly, `41` `E402` /late-import notices stem primarily from

`fastapi/__init__.py` declaring `__version__` prior to its `20` explicit PEP 484 `from .module import X as X` re-exports. Zero wildcard imports (`from ... import *`) exist anywhere in `fastapi/`.

3. **Pioneering PEP 727 Inline Parameter Documentation (`25.2%` Block Docstrings)**: Rather than maintaining fragile multi-hundred-line `:param:` docstring blocks that drift from function signatures (`doc_signature_mismatches = 0`, `empty_docstrings = 0`), FastAPI pioneered PEP 727 `Annotated[T, Doc("...")]` via `annotated-doc`. Every parameter on `FastAPI`, `APIRouter`, `Path`, `Query`, `Body`, `Header`, `Cookie`, `Depends`, and `Security` carries co-located Markdown documentation directly inside its type annotation, surfacing rich tooltips in modern Python language servers and API reference generators.

5.2 SIG Maintainability, Coupling & 12-Factor Operational Readiness

Evaluated against Software Improvement Group (SIG) maintainability benchmarks and 12-Factor Production Readiness Review (PRR) checks, FastAPI achieves zero mutable global state, zero missing I/O timeouts, and zero unstructured root-logger side effects, while concentrating algorithmic branching inside five deliberate compilation and ASGI dispatch engines.

Engineering Health Dimension	Metric Key	Count	Assessment & Codebase Grounding
SIG Unit Complexity / Size	`complex_functions_count`	`88`	Concentrated in 5 core ASGI/OpenAPI/DI engines; public route decorators have `cognitive = 0` (long `Doc()` signatures)
SIG Unit Interfacing (`> 6` params)	`excessive_parameters_count`	`52`	Intentional ergonomic OpenAPI keyword arguments (`FastAPI.__init__` = `37` params; `@app.get` / `post` / `put` / `delete` = `23` params)
Mutable Global State	`global_state_usages_count`	`0`	State is encapsulated on `FastAPI` instances and per-request ASGI `scope` dicts
Runtime Circular Dependencies	`circular_dependencies_count`	`1`	`fastapi._compat.v2 -> fastapi.params -> fastapi.datastructures -> fastapi._compat.v2` (`+2` cleanly `TYPE_CHECKING`-guarded)
Bug-Risk Idioms	`bug_risk_idioms_count`	`4`	`1` mutable default `values={}` (`_compat/v2.py:176`, unmutated) + `3` benign control-flow `except ...: pass` blocks (`routing.py`)

Engineering Health Dimension	Metric Key	Count	Assessment & Codebase Grounding
Optional / Transitive Imports	`undeclared_dependencies_count`	`16`	Transitive core packages (`pydantic_core` , `anyio`) & guarded extras (`python_multipart` , `email_validator` , `fastapi_cli` , `ujson` , `orjson`)
Python Target Compatibility	`language_version_mismatches_count`	`0`	Clean `requires-python = ">=3.10"` compatibility across Python `3.10` – `3.14` (`keep-runtime-typing = true`)
12-Factor I/O Resilience	`missing_io_timeouts_count`	`0`	Pure ASGI message passing without unguarded blocking socket/HTTP calls
12-Factor Config Lookups	`scattered_env_access_count`	`9`	`7` in `fastapi/telemetry/` (`OTEL_*` auto-config) + `2` in `applications.py` / `routing.py`
12-Factor Structured Logging	`unstructured_logging_count`	`0`	Named `logging.getLogger("fastapi")` in `fastapi/logger.py` with zero `basicConfig` hijacking

Cognitive Complexity Concentration in Core Compilers

Whereas `FastAPI.get` , `post` , `put` , and `delete` appear in AST length scans solely due to comprehensive `Annotated[..., Doc()]` signatures (`cognitive = 0`), genuine branching complexity concentrates in five high-leverage internal routines:

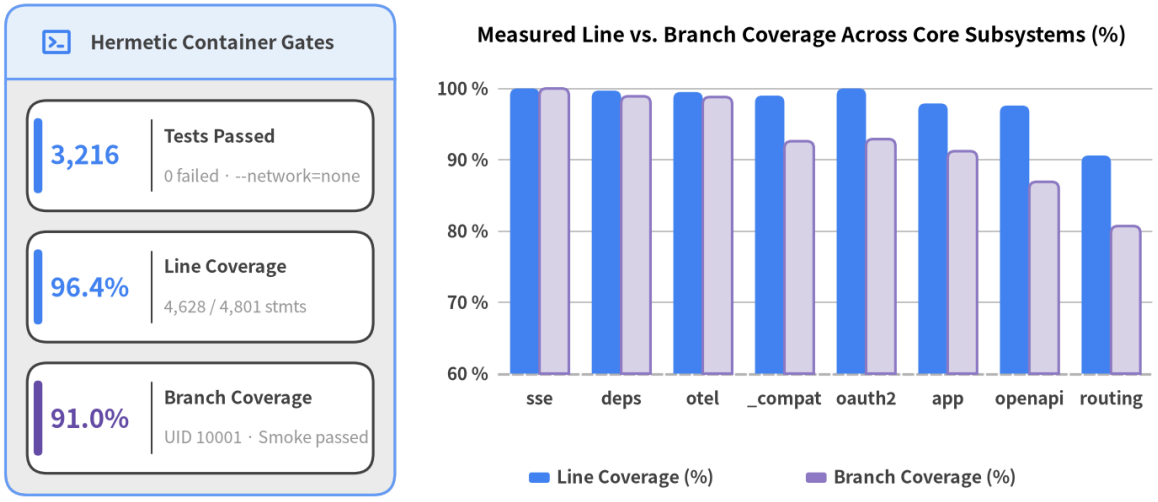
Hotspot Callable	Location	Cognitive Complexity	Length / Branches	Architectural Role & Trade-Off
`get_request_handler`	`fastapi/routing.py:397`	`159` *(inner `app` : `157`)*	`408` LOC / `49` br	Pre-compiles body extraction (JSON/form/multipart), DI execution, sync/async dispatch, SSE/JSONL streaming, and Pydantic response serialization into a single hot-path ASGI closure
`get_openapi_path`	`fastapi/openapi/utils.py:303`	`109`	`230` LOC / `29` br	Synthesizes OpenAPI 3.1 operation metadata, parameter schemas, `requestBody` media types, security scopes, callbacks, and status-code responses
`NativeTelemetry.__call__`	`fastapi/telemetry/_asgi.py:228`	`85`	`230` LOC / `28` br	Low-overhead ASGI middleware managing HTTP/WebSocket span lifecycles, header propagation, route templates, and exception events
`analyze_param`	`fastapi/dependencies/utils.py:409`	`64`	`196` LOC / `29` br	Introspects `Annotated` and default `FieldInfo` descriptors via `typing-inspection` to classify path, query, header, cookie, body, and `Depends` inputs

Hotspot Callable	Location	Cognitive Complexity	Length / Branches	Architectural Role & Trade-Off
`solve_dependencies`	`fastapi/dependencies/utils.py:643`	`33`	`143` LOC / `20` br	Executes the recursive `Dependant` DAG per request with `(call, scopes)` caching and `AsyncExitStack` registration for `yield` teardown

> [!TIP] **Performance-Conscious Refactoring Opportunity**: `get_request_handler` intentionally hoists route metadata checks outside per-request execution by returning a specialized `async def app(request)` closure. Factoring internal phases (`_parse_request_body` , `_execute_solved_endpoint` , `_build_validated_response`) into dedicated private helpers can reduce cognitive complexity toward `<= 25` without adding runtime indirection.

5.3 Hermetic Container Runtime Verification & Subsystem Coverage

Containerized verification (`container_runtime_audit` : **97/100, Grade S**) executed inside an air-gapped Docker container (`--network=none` , non-root `UID 10001` , CPython `3.12.12` , `.pni/dockerfiles/root/Dockerfile`) passed every runtime gate cleanly on the first pass.



Hermetic Container Verification KPIs & Measured Line vs. Branch Coverage Across Core Subsystems

Runtime Verification Summary (`RV-01` – `RV-04`)

Verification Gate	Status	Duration	Evidence & Configuration Details
`RV-01` **Harness Image Build**	`PASSED`	`0.79s` *(cached)*	`.pni/dockerfiles/root/Dockerfile` (`PYTHON_VERSION=3.12` , non-root `UID 10001` , isolated `pni-venv`)
`RV-02` **Air-Gapped Test Suite**	`PASSED`	`265.57s`	`3,228` collected: `3,216` **passed**, `0` **failed**, `0` **errors**, `12` skipped under `--network=none`
`RV-03` **Measured Coverage**	`PASSED`	*(with `RV-02`)*	`96.4%` **line** (`4,628 / 4,801` stmts) and `91.0%` **branch** (`1,419 / 1,560` branches) via `pytest-cov`
`RV-04` **Library Smoke Check**	`PASSED`	`3.36s`	Clean zero-side-effect import (`importlib.import_module("fastapi")` -> `"Imported fastapi cleanly"`)

FastAPI's `pyproject.toml` test harness enforces exceptional rigor (`python_test_audit` : **95/100, Grade S**): `--strict-config` , `--strict-markers` , `strict_xfail = true` , `filterwarnings = ["error"]` , `timeout = "20"` , and `inline-snapshot` verification of generated OpenAPI 3.1 schemas. Static test inspection identified only `2` assertion-free test functions across `1,835` test definitions (`tests/test_multipart_installation.py:141` , where `warnings.catch_warnings(record=True)` is not bound or asserted, and `tests/test_webhooks_security.py:34`).

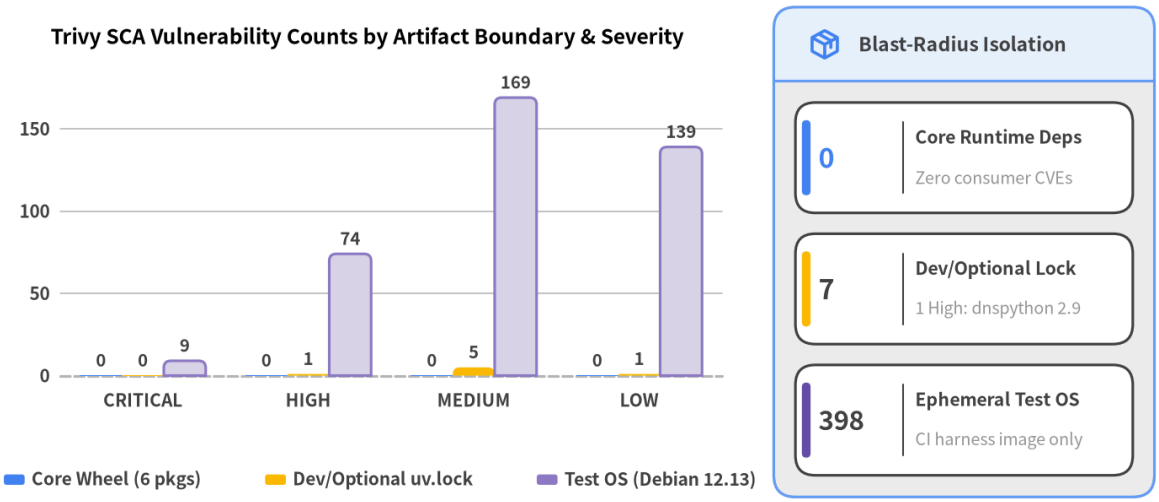
Lowest-Covered Modules (Measured Ascending Order)

Examining the bottom of the coverage distribution confirms that core runtime logic is covered above `90%`, while `0.0%` entries consist solely of 1–2 line Starlette re-export shims exercised by `docs_src/` tutorial integration suites (which `[tool.pytest]` excludes during core unit runs via `--ignore=docs_src`):

Module Path	Line Coverage	Branch Coverage	Covered / Total Stmts	Missing Lines	Root-Cause Engineering E
`fastapi/__main__.py`	`0.0%`	—	`0 / 2`	`[1, 3]`	2-line `python -m fastapi` entrypoint delegating direc `fastapi.cli:main`
`fastapi/middleware/cors.py`	`0.0%`	—	`0 / 1`	`[1]`	1-line Starlette `CORSMiddle` convenience re-export shir
`fastapi/middleware/gzip.py`	`0.0%`	—	`0 / 1`	`[1]`	1-line Starlette `GZipMiddle` convenience re-export shir
`fastapi/middleware/httpsredirect.py`	`0.0%`	—	`0 / 1`	`[1]`	1-line Starlette `HTTPSRedirectMiddleware` convenience re-export shir
`fastapi/middleware/trustedhost.py`	`0.0%`	—	`0 / 1`	`[1]`	1-line Starlette `TrustedHostMiddleware` convenience re-export shir
`fastapi/templating.py`	`0.0%`	—	`0 / 1`	`[1]`	1-line Starlette `Jinja2Temp` convenience re-export shir
`fastapi/responses.py`	`86.7%`	—	`26 / 30`	`[65, 66, 95, 96]`	`@deprecated` `UJSONResponse.render` & `ORJSONResponse.render` b skipped when optional `ujson` / `orjson` C extensic absent
`fastapi/cli.py`	`87.5%`	`50.0%`	`7 / 8`	`[13]`	`cli_main()` delegation bra external `fastapi-cli`; cor mocks `cli_main = None` t clean `RuntimeError` fallba

6. Deep Dive: Security, Supply Chain & API Surface

FastAPI `v0.162.2` maintains an exemplary framework security posture (`security_vulnerability_audit`: **91/100, Grade S**; `security` axis: **92/100**): all `6` core runtime dependencies carry **zero CVEs**, deterministic SAST scanning across `52` source modules (`19,837` LOC) detects **zero CWE-classified dangerous execution sinks**, and Trivy + AST scans confirm **zero hardcoded secrets** and **zero container IaC misconfigurations**. Vulnerability findings are strictly isolated to `7` transitive advisories in the development/optional `uv.lock` (`1` HIGH in `dnspython@2.8.0 -> 2.9.0`) and `398` OS-layer advisories inside the ephemeral `debian 12.13` CI test image.



Vulnerability Isolation Across Core FastAPI Wheel, Dev/Optional uv.lock, and Ephemeral Debian 12.13 Test OS

6.1 Supply-Chain Hygiene, Packaging & Governance Baseline

FastAPI's PEP 621 `pyproject.toml` manifest and dependency architecture (`dependencies` axis: **96/100**) minimize downstream supply-chain exposure by keeping the default `pip install fastapi` footprint ultra-lean while isolating optional runtime integrations (`[standard]` , `[opentelemetry]` , `[all]`) and PEP 735 `[dependency-groups]` (`dev` , `tests` , `docs` , `translations` , `github-actions`) behind a reproducible `uv.lock` .

Supply-Chain & Governance Gate	Metric / Value	Verdict	Architectural & Compliance Evidence
Deterministic Lockfile (`has_lockfile`)	`True` (`uv.lock`)	`PASS`	Reproducible multi-group dependency graph committed under `uv`
Core Runtime Dependencies (`dependencies_count`)	`6` packages	`PASS`	Ultra-lean ASGI, validation, typing, inline docs, and OTel API surface
Unpinned Specifiers (`unpinned_dependencies_count`)	`0`	`PASS`	Every dependency declares an explicit SemVer `>=` floor without upper-cap lock-in
Dual Licensing Clarity (`has_license`)	`True`	`PASS`	Repository Apache-2.0 `LICENSE` + upstream FastAPI MIT `LICENSE-FASTAPI`
Committed / Hardcoded Secrets	`0` **files /** `0` **literals**	`PASS`	Clean across both Trivy secret scanner and deterministic AST literal inspection
AST Dangerous Sinks (`ast_dangerous_sinks_count`)	`0`	`PASS`	Zero `CWE-78` / `89` / `94` / `502` / `22` execution or deserialization sinks in `fastapi/`
Standalone HTTP Endpoints (`api_endpoints_count`)	`0`	`PASS`	Reusable `library` archetype; binds zero network ports upon import
Repo Governance (`SECURITY.md` / **Dependabot)**	`False` / `False`	`WARN`	Omitted in this standalone showcase snapshot (upstream FastAPI publishes GitHub Security Advisories)

Core Wheel Runtime Dependency Inventory (`0` CVEs)

Core Package	`pyproject.toml` Constraint	Resolved `uv.lock`	Primary Architectural Responsibility	Trivy CVEs
`starlette`	`>=0.46.0`	`1.7.0`	Asynchronous ASGI 3.0 routing, lifespan, WebSockets, middleware & `TestClient`	`0`
`pydantic`	`>=2.9.0`	`2.13.5`	Rust-backed `pydantic-core` `TypeAdapter` validation, serialization & JSON Schema	`0`

Core Package	`pyproject.toml` Constraint	Resolved `uv.lock`	Primary Architectural Responsibility	Trivy CVEs
`typing-extensions`	`>=4.8.0`	`4.16.0`	Backported typing constructs (`Annotated`, `@deprecated`, `TypedDict`, `TypeAliasType`)	`0`
`typing-inspection`	`>=0.4.2`	`0.4.4`	High-speed runtime `Annotated` and generic signature introspection for `Dependant` DAGs	`0`
`annotated-doc`	`>=0.0.2`	`0.0.5`	Zero-overhead PEP 727 `Doc("...")` inline parameter documentation carrier	`0`
`opentelemetry-api`	`>=1.44.0`	`1.45.1`	Vendor-neutral distributed tracing & metrics API (`no-op` until an OTel SDK is configured)	`0`

6.2 Vulnerability Triage: Core Wheel vs. `uv.lock` vs. Test Harness OS

Containerized Trivy `0.69.3` scanning (`engine=docker`, `scanned_filesystem=True`, `scanned_resolved_image=True` against `.pni/dockerfiles/Dockerfile`) flagged an overall scan status of `failed` due to `405` combined advisories (`7` application lockfile + `398` OS base-image). Separating these results by deployment boundary is essential for accurate risk assessment:

Artifact Scope	CRITICAL	HIGH	MEDIUM	LOW	Total	Downstream Consumer Impact & Remediation
1. FastAPI Core Runtime Wheel (`6` pkgs)	`0`	`0`	`0`	`0`	`0`	**None**: Downstream `fastapi` installations resolve clean latest SemVer-compatible packages
2. Transitive Dev / Optional `uv.lock`	`0`	`1`	`5`	`1`	`7`	**Dev/CI Only**: `dnspython@2.8.0` (via optional `email-validator`); upgrade via `uv lock --upgrade-package dnspython` (`>=2.9.0`)
3. Ephemeral Test Harness OS (`debian 12.13`)	`9`	`74`	`169`	`139`	`398`	**None (Never Shipped)**: Air-gapped `bookworm-slim` pytest runner; refresh via `apt-get upgrade -y` or `trixie-slim`

Top Pre-Sorted CRITICAL & HIGH Vulnerability Advisories

The table below renders the exact pre-sorted top `CRITICAL` and `HIGH` Trivy findings across `uv.lock` and the `debian 12.13` test-harness container image, including exact installed and upstream fixed versions:

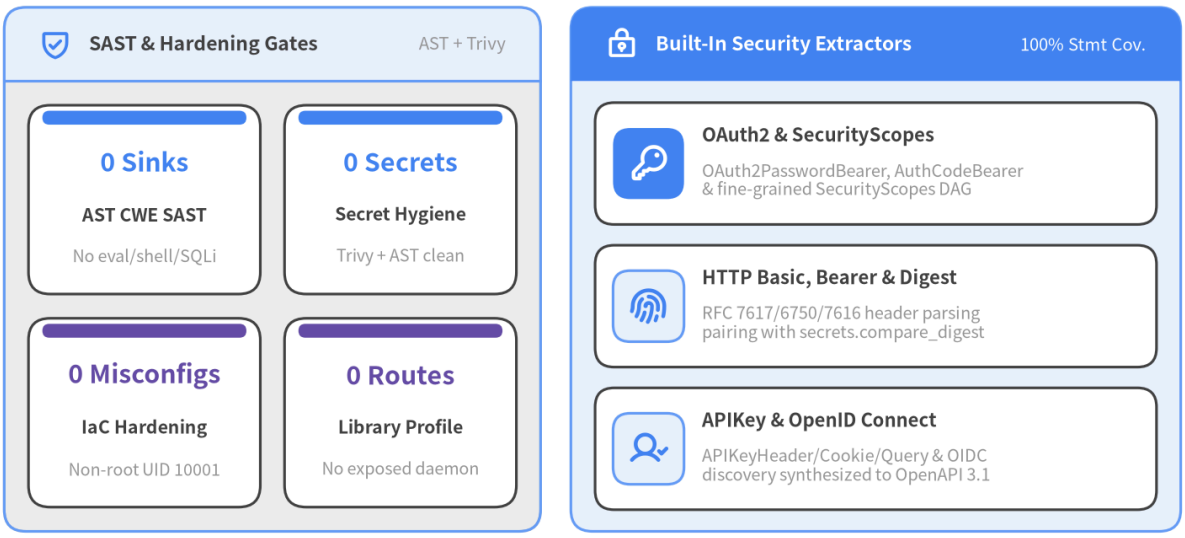
Severity	CVE ID	Package & Installed Version	Ecosystem	Fixed Version	Advisory Summary
`CRITICAL`	`CVE-2026-42010`	`libgnutls30@3.7.9-2+deb12u5`	`debian`	`3.7.9-2+deb12u7`	`gnutls`: Authentication Bypass via NUL Character in Username
`CRITICAL`	`CVE-2026-31789`	`libssl3@3.0.18-1~deb12u2`	`debian`	`3.0.19-1~deb12u2`	`openssl`: Heap buffer overflow on 32-bit systems from large X.509 certificate processing
`CRITICAL`	`CVE-2026-31789`	`openssl@3.0.18-1~deb12u2`	`debian`	`3.0.19-1~deb12u2`	`openssl`: Heap buffer overflow on 32-bit systems from large X.509 certificate processing
`CRITICAL`	`CVE-2026-8376`	`perl-base@5.36.0-7+deb12u3`	`debian`	`5.36.0-7+deb12u4`	`perl`: Heap buffer overflow when compiling regular expressions on 32-bit builds
`CRITICAL`	`CVE-2023-45853`	`zlib1g@1:1.2.13.dfsg-1`	`debian`	*(no upstream fix)*	`zlib`: Integer overflow and resultant heap-based buffer overflow in `zipOpenNewFileInZip4_6`
`CRITICAL`	`CVE-2026-33845`	`libgnutls30@3.7.9-2+deb12u5`	`debian`	`3.7.9-2+deb12u7`	`gnutls`: Denial of Service via DTLS zero-length fragment
`CRITICAL`	`CVE-2025-7458`	`libsqlite3-0@3.40.1-2+deb12u2`	`debian`	*(no upstream fix)*	`sqlite`: SQLite integer overflow
`CRITICAL`	`CVE-2026-13221`	`perl-base@5.36.0-7+deb12u3`	`debian`	`5.36.0-7+deb12u4`	`perl`: Incorrect regular expression processing via large regular expressions
`CRITICAL`	`CVE-2026-42496`	`perl-base@5.36.0-7+deb12u3`	`debian`	`5.36.0-7+deb12u4`	`perl-archive-tar`: Path traversal via crafted symlinks allows arbitrary file access
`HIGH`	`CVE-2026-114605`	`dnspython@2.8.0`	`uv`	`2.9.0`	`dnspython`: Denial of Service via excessive DNS compression pointer traversal (`CVSS 7.5`)
`HIGH`	`CVE-2026-45447`	`libssl3@3.0.18-1~deb12u2`	`debian`	`3.0.20-1~deb12u2`	`openssl`: Heap Use-After-Free in OpenSSL `PKCS7_verify()`

Severity	CVE ID	Package & Installed Version	Ecosystem	Fixed Version	Advisory Summary
HIGH	CVE-2026-16742	libsystemd0@252.39-1~deb12u1	debian	*(no upstream fix)*	systemd : systemd-homed local privilege escalation via missing home-record signature check
HIGH	CVE-2026-16742	libudev1@252.39-1~deb12u1	debian	*(no upstream fix)*	systemd : systemd-homed local privilege escalation via missing home-record signature check
HIGH	CVE-2026-45447	openssl@3.0.18-1~deb12u2	debian	3.0.20-1~deb12u2	openssl : Heap Use-After-Free in OpenSSL PKCS7_verify()
HIGH	CVE-2026-76844	zlib1g@1:1.2.13.dfsg-1	debian	*(no upstream fix)*	webpack-dev-middleware / zlib1g metadata match: Information Disclosure via Path Traversal

> [!NOTE] **Container IaC Hardening (SEC-02)**: Trivy reported 0 **Dockerfile misconfigurations** (misconfigurations = []). The test harness pins ghcr.io/astral-sh/uv:python3.12-bookworm-slim by SHA-256 digest, provisions unprivileged user pni (UID 10001), and runs offline (--network=none). Active .pni/pni.toml design waiver DS-0026 appropriately exempts batch pytest containers from daemon HEALTHCHECK directives.

6.3 AST Dangerous Code Sinks (CWE) & Built-In Security Architecture

Because FastAPI is an installable framework library (runtime_types = ["library"]), it exposes 0 **standalone HTTP endpoints** (api_endpoints = []) while providing downstream applications with strongly typed, dependency-injected security extractors across fastapi/security/* (all achieving 100.0% **statement coverage** under hermetic testing).



Zero-Sink AST SAST Verification Gates & FastAPI Built-In Security Extractor Primitives

---

CWE SAST Audit & Built-In `fastapi/security/*` Subsystems

1. **Zero CWE Execution Sinks (`SEC-04`)**: Deterministic AST scanning across all `52` source files verified complete absence of SQL string interpolation (`CWE-89`), shell command execution (`CWE-78`, zero `os.system` or `subprocess(..., shell=True)`), dynamic code evaluation or unsafe deserialization (`CWE-94` / `CWE-502`, zero `eval`, `exec`, `pickle`, or unsafe `yaml.load`), and unvalidated file traversal (`CWE-22`; `fastapi/routing.py` normalizes mounted SPA static paths via `os.path.realpath` containment checks before serving files).
2. **Co-Located Authorization & OpenAPI 3.1 Contract Generation (`SEC-05`)**: Every security class subclassing `SecurityBase` (`fastapi/security/base.py`) acts simultaneously as an async `Depends()` / `Security()` request validator and an OpenAPI 3.1 `securitySchemes` descriptor—ensuring authorization rules enforced at runtime never drift from published API documentation:

Security Module	Exported Primitives	Runtime Enforcement & OpenAPI 3.1 Integration	Measured Stmt Cov.
`fastapi/security/oauth2.py`	`OAuth2`, `OAuth2PasswordBearer`, `OAuth2AuthorizationCodeBearer`, `OAuth2PasswordRequestForm` (`Strict`), `SecurityScopes`	Extracts Bearer tokens, parses RFC 6749 form credentials, and propagates required `Security(..., scopes=[...])` across nested `Dependant` sub-trees	`100.0%` (`66/66`)
`fastapi/security/http.py`	`HTTPBasic`, `HTTPBasicCredentials`, `HTTPBearer`, `HTTPDigest`, `HTTPAuthorizationCredentials`	Strictly validates `Authorization` schemes, handles malformed Base64/ASCII payloads with `401 WWW-Authenticate` headers, and structures credentials for constant-time `secrets.compare_digest` checks	`100.0%` (`81/81`)
`fastapi/security/api_key.py`	`APIKeyHeader`, `APIKeyCookie`, `APIKeyQuery`	Enforces required or optional (`auto_error=False`) API keys across headers, cookies, and query parameters	`100.0%` (`42/42`)
`fastapi/security/open_id_connect_url.py`	`OpenIdConnect`	Binds OIDC discovery URLs into OpenAPI 3.1 `openIdConnect` security definitions and validates `Authorization` headers	`100.0%` (`21/21`)